

**CatSight AI: Enhancing Document Search Accuracy and User Interaction through OCR,
Vector Similarity Search, and LLM-Based Chat Interfaces**

A Thesis Project

Submitted to

**The Faculty of the Department of Computer Science
Mindanao State University – Iligan Institute of Technology
In Fulfillment of the Requirements in CSC198 – Research Method**

Arellano, Carlo P.

Carnaje, Michael James M.

Lavesores, Fulgent Kvasir E.

Proponents

Professor Dante D. Dinawanao

Adviser

How to Read This Comparison

This document is the revised thesis with every change against the defended edition marked.

- **Blue text** was added or rewritten in the revision.
- ~~Red, struck-through text~~ was removed; a removed heading appears as a struck-through line in bold.
- A bar in the margin marks every changed paragraph, figure, table and algorithm: scan the margins to find the changes.
- New figures and screenshots have blue captions and a blue frame; removed figures are shown small and crossed out.

Chapters 1 and 2 changed little: the specific objectives, the scope, and corrections to citations.

Chapters 3 to 7 hold most of the revisions requested by the panel. The Revision Summary that follows lists each revision with the section and page that address it.

Revision Summary

The table lists the revisions requested by the panel after the defense and where each one is addressed in this edition. Page numbers in the second column refer to the defended manuscript; the last column refers to this document. The panel’s numbering, which skips 9, is kept.

No.	Revision requested	Section / page (defended)	Action taken (this edition)
1	Missing Theoretical Framework	Methodology, p. 34	Completed Chapter 3: retrieval theory, LLM concepts, the evaluation measures (Section 3.3, previously empty), TAM (Section 3.4, previously only mentioned), and the conceptual framework in Figure 3.1 (p. 43).
2	Proper and consistent figure sizes; make them clearer	Methodology, p. 40	All diagrams redrawn as vector graphics with one style and font; all screenshots captured at the same window size and shown at full text width.
3	Add Design and Implementation Issue	Methodology, p. 34	Wrote Chapter 5: data model, data structures, and fifteen issues with their solutions (Table 5.1, p. 78).
4	Proper spacing	Methodology, p. 44	Typeset in $\LaTeX$ with uniform double spacing, heading spacing and paragraph indentation.
5	Missing summarization figure flow	Methodology, p. 44	Added Figure 4.5 (p. 62).
6	Use ROUGE, BLEU and BERTScore for summarization metrics	Methodology	Defined in Section 3.3.2 (p. 39) and applied in Section 6.5.
7	Mention the figures that are shown	Methodology	Every figure, table and algorithm is referred to in the text before it appears.
8	Use $\LaTeX$ equations	Methodology	The paper is written in $\LaTeX$; Equations 3.1–3.14 are numbered and referenced.
10	Provide cosine similarity score equations	–	Equations 3.1–3.3 in Section 3.1.3 (p. 33).
11	Explain token and cosine similarity score	–	Section 3.1.1 (search tokens and model tokens) and Section 3.1.3 (score range, threshold, worked example).
12	Discuss map-reduce in the paper	–	Section 3.2.4 with Equations 3.6–3.7, and Section 5.3.8.
13	Write an algorithm for generating summaries	–	Algorithm 3 (p. 60).
14	Define hallucination in LLMs and its basis	–	Section 3.2.2: definition, types, causes, and mitigation; implementation in Section 5.3.10.
15	Explain the zero-shot method	–	Section 3.2.1 (p. 35).

No.	Revision requested	Section / page (defended)	Action taken (this edition)
16	Make the specific objectives more concise	Specific Objective, p. 4	Rewritten in Section 1.3.2 (p. 4).
17	Show an architecture diagram of how the components interact	3.2 Methodology, p. 37	Figure 4.3 with numbered interactions (p. 58), and the sequence of a chat request in Figure 4.8.
18	Express the diagrams as algorithms instead of flowcharts	–	The processes are given as Algorithms 1–6, replacing the defended flowchart. The other figures show structure and interactions; the summarization flow requested in revision 5 accompanies Algorithm 3.

Abstract

The CatSight AI system addresses the limitations of MSU-IIT’s keyword-based document search by ~~leveraging a unified pipeline of OCR~~ combining optical character recognition (OCR), vector embeddings, and ~~an LLM-driven conversational interface~~. ~~The architecture a conversational interface driven by a large language model (LLM).~~ It follows a retrieval-augmented generation (RAG) ~~framework: documents (design: each issuance, scanned or digital) are processed via OCR (ensuring text extraction from scanned or image-based documents), segmented into chunks, embedded into vector representations stored in a vector database, and then retrieved to support LLM response generation. This integration enables users to issue intuitive natural language queries and receive contextually relevant answers through a chat-based interface.~~

~~In evaluations against the legacy system, CatSight AI achieved higher retrieval accuracy, semantic relevance, and efficiency. The hybrid approach substantially improves semantic understanding, capturing paraphrases and synonyms (for example, equating “academic calendar” with “term schedule”) that the keyword system misses, and retrieves more relevant results than keyword-only search. User testing confirmed a marked usability gain: CatSight,~~ is converted to text page by page with Marker, whose Surya OCR 2 model reads every page while the Qwen3-VL vision-language model corrects tables, forms, and handwriting; the text is catalogued zero-shot, with map-reduce for long documents; and it is split into passages indexed for both semantic (cosine similarity on BGE-M3 embeddings) and keyword search. A question is answered by an agent that searches the library, fuses the two result lists with Reciprocal Rank Fusion, reranks them, and writes an answer that cites the exact pages it used. All models are open-weight: they are reached through OpenRouter because the proponents have no GPU server, but the institution can host the same weights on its own hardware. On a sample of seven documents, Marker with Qwen3-VL recovered 91% of the facts read from the page images, without altering names or dates, while the repositories’ own text layers yielded 40%

and nothing at all from scanned pages. In the usability evaluation of the defended prototype, CatSight AI's System Usability Scale score averaged 77.17%, ~~versus only~~, against 50.33 % for the existing system. ~~Overall, these~~ These results indicate that CatSight AI ~~outperforms traditional methods in search accuracy and user experience, offering~~ offers a more accurate, intelligent, and efficient ~~document retrieval solution~~ way to retrieve MSU-IIT's documents than traditional keyword search.

Keywords: ~~Document Retrieval System~~document retrieval system, ~~Optical Character Recognition (OCR)~~optical character recognition, ~~Vector-Based Similarity Search~~retrieval-augmented generation, ~~Large Language Model (LLM)~~vector similarity search, ~~Semantic Search~~large language model, open-weight models

List of Figures

3.1	Conceptual Framework of the Study	43
4.1	Steps for Building LLM-Powered Applications, Adapted from Sinha (2024); Steps 1–6 Were Adopted	45
4.2	How a Large Language Model Generates Text, After Mulugeta (2024)	52
4.3	System Architecture of CatSight AI	58
4.4	Stages of the Document Pipeline	59
4.5	Summarization Flow: One Structured Call, or Map-Reduce for Long Text	62
4.6	Hybrid Search-Retrieval Flow	62
4.7	The Chat Agent as a State Graph	63
4.8	Sequence of One Chat Question	64
4.9	Landing Page	65
4.10	Sign-In Page with One-Click Guest Access	65
4.11	Dashboard	66
4.12	Upload Dialog with Three Special Orders Selected	67
4.13	The Ingestion Pipeline on the Dashboard While the Library Is Processed	67
4.14	A Document at the Reading Stage	68
4.15	Document Library	68
4.16	Document Overview: the Catalogue Generated from the Document	69
4.17	The Original PDF	70
4.18	Text Extracted by Marker	70
4.19	Indexed Passages with Their Page Numbers	71
4.20	Chat with Suggested Questions	71
4.21	A Cited Answer	72
4.22	A Citation Opened to Its Source Passage	72
4.23	Tags	73
4.24	Settings	73
4.25	Chat and Dashboard on a Phone	74
5.1	Data Model of CatSight AI	77
6.1	Share of Facts Recovered by Each Text-Extraction Setup	86
6.2	System-Workflow The Library by Decade of Issuance and by Tag	88
6.3	Search by Reference Number: “SO 1592” Finds Special Order No. 01592-IIT	89
6.4	Search by Meaning: a Paraphrased Query Finds the Grants-in-Aid Resolution	89
6.5	A Question in the Search Page, with a Short Cited Answer	90
6.6	A Follow-Up Question Answered from the Conversation	92
6.7	A Chat Limited to One Document	92
6.8	A Question in Filipino Answered in Filipino	93

List of Algorithms

1	Document Processing	59
2	Page Extraction with Marker	60
3	Generate Summary (Cataloguing with Map-Reduce)	60
4	Document Indexing	61
5	Hybrid Search-Retrieval	61
6	Interactive Chat	63

Chapter 1 Research Description

In today's era of digital transformation, organizations generate and store an unprecedented volume of documents daily. This exponential growth in document storage has created a pressing demand for efficient document retrieval systems (DRS) to access, manage, and leverage information effectively. A document retrieval system is a specialized application of information retrieval (IR) designed to locate relevant documents within vast repositories based on user queries. By enabling quick and accurate access to information, DRS plays a pivotal role in knowledge management, informed decision-making, and productivity enhancement across diverse domains, including education, business, healthcare, and research (Blair ~~, n.d.~~; ~~Maedonald & Tait, 2020~~ & Maron, 1985; Manning et al., 2008).

At its core, document retrieval addresses the challenge of identifying stored documents that contain pertinent information. The primary objective is to distinguish between relevant and non-relevant documents for a user's specific query. Modern DRS employs various techniques to achieve this, ranging from traditional keyword-based methods to advanced machine learning approaches. Keyword-based retrieval is a foundational technique that matches user queries to specific keywords within documents. While it is straightforward and easy to implement, its reliance on exact matches can limit its ability to capture the broader context of queries (Blair ~~, n.d.~~ & Maron, 1985). Boolean retrieval, an early and widely used approach, utilizes logical operators like AND, OR, and NOT to narrow down search results based on exact keyword matches. Though simple, it often lacks the flexibility to handle nuanced user queries (~~Maedonald & Tait, 2020~~ Manning et al., 2008). The Vector Space Model (VSM) improves on this by representing documents and queries as vectors in a multi-dimensional space, with relevance determined by the cosine similarity between these vectors, enabling more precise ranking of results compared to Boolean methods (Blair ~~, n.d.~~ & Maron, 1985). Latent Semantic Analysis (LSA) takes this further by uncovering hidden relationships between terms in a document collection through term-document matrix analysis, thereby enhancing retrieval accuracy by capturing the semantic meaning of words rather than relying solely on exact matches (~~Maedonald &~~

Tait, 2020Manning et al., 2008). Machine learning-based retrieval techniques, such as support vector machines (SVMs), random forests, and deep learning models, dynamically adapt to user behavior and context, significantly enhancing retrieval accuracy (Blair, 1984). Neural information retrieval systems, leveraging advanced deep learning architectures like recurrent neural networks (RNNs) and transformers, model complex language patterns and contextual nuances in documents, offering even greater precision and relevance in search results (Maedonald & Tait, 2020Zhu et al., 2023).

A notable advancement in modern DRS is the integration of Retrieval-Augmented Generation (RAG) frameworks. RAG combines document retrieval with generative AI models to provide highly contextual and human-like responses. This approach retrieves relevant documents from the repository and feeds them into generative models, like GPT, which synthesize coherent and informative answers to user queries. The RAG framework not only enhances the retrieval process but also bridges the gap between static document retrieval and dynamic information synthesis, making it a revolutionary tool in data-driven environments (Maedonald & TaitLewis et al., 2020).

This research aims to explore the evolving landscape of document retrieval systems, emphasizing their significance in transforming data into actionable knowledge. By analyzing advanced retrieval techniques and highlighting the role of RAG, this study underscores the pivotal role of DRS in fostering innovation and informed decision-making in the digital age.

1.1 Background of the Study

Effective management and access to extensive information repositories are crucial for institutions that maintain vast collections of documents. Document Retrieval Systems (DRS) are essential tools that enable users to efficiently locate specific documents. This study focuses on the Mindanao State University – Iligan Institute of Technology (MSU-IIT), which has its own repositories for Special Orders, Memorandums, and other important records. MSU-IIT maintains two primary repositories:

- **Board of Regents (BOR) Resolutions Archive:** This repository contains BOR resolutions of the MSU System. (Mindanao State University - Iligan Institute of Technology, n.d.-a)

1.2 Statement of the Problem

Traditional keyword-based document retrieval systems face global challenges in providing accurate and relevant results due to their lack of contextual and semantic understanding. This often leads to inefficient search experiences for users worldwide. At MSU-IIT, students, faculty, and staff experience similar issues when retrieving institutional documents. The current system's reliance on exact keyword matches limits access to important information. A more advanced, context-aware solution is needed to address these limitations and position MSU-IIT as a model for modern document retrieval.

1.3 Research Objective

The following are the general and specific ~~objects~~: objectives of the study.

1.3.1 General Objective

To develop and evaluate a content-based document retrieval system that integrates Optical Character Recognition (OCR), vector embeddings, and Large Language Model (LLM)-powered conversational interaction.

~~Specific Objective~~

1.3.2 Specific Objectives

Specifically, the study aims:

1. ~~To collect and annotate representative documents from MSU-IIT's institutional repositories, specifically to~~ collect a representative set of 50 issuances from IIT Docs and ~~BOR Resolutions~~: the BOR Resolutions archive;
2. ~~To design a content-based document retrieval system architecture integrating OCR for scanned PDFs, vector embeddings for semantic~~ to design an architecture that combines OCR, hybrid (semantic and keyword) search, and ~~LLMs for conversational queries~~: an LLM chat agent;

3. ~~To implement the system's key components: OCR processing, semantic search using vector similarity, and an LLM-based chat interface.~~ to implement and deploy the document pipeline, the search, and the cited chat interface;
4. ~~To to~~ evaluate the system's ~~performance in terms of accuracy, user experience, and retrieval speed.~~ retrieval accuracy, summary quality, response time, and usability; and
5. ~~To analyze evaluation results to validate system effectiveness and recommend future enhancements~~ to analyze the results and recommend improvements.

1.4 Scope and Limitations of the Research

This study focuses on enhancing the Document Retrieval System (DRS) at Mindanao State University --Iligan Institute of Technology (MSU-IIT). ~~The research will involve~~ It uses a sample of 50 PDF documents ~~and will be conducted from January 2025 to April 2025, with participants including~~ from IIT Docs and the BOR Resolutions archive. The first prototype was developed and evaluated with MSU-IIT students, faculty, and staff ~~. A notable limitation of the current system is its inability to analyze images and tables within documents, restricting its effectiveness in processing non-textual data. This constraint may impact the retrieval accuracy for documents where critical information is presented in these formats. By acknowledging this limitation, from~~ January to April 2025. After the defense, the system was rebuilt on the same design and re-deployed in October 2026; this revised paper describes the rebuilt system (Chapter 4) and states for each result which version it was measured on (Chapter 6).

The study has the following limitations:

- **Non-textual content.** The OCR step transcribes text and tables into Markdown, but it does not describe photographs, diagrams or seals; signatures are recorded only as [signature]" and unreadable handwriting as [illegible]". Information carried only by images cannot be retrieved.

- **Corpus size.** Fifty documents (48 after removing exact duplicates) are enough to test the pipeline and the user experience, but not to measure performance at the scale of the full repositories.
- **Model hosting.** All models are open-weight, but because the proponents have no GPU server, the language, embedding, and reranking models were reached through OpenRouter, which hosts the same models; only the OCR model ran on the study's own server. A fully local configuration with Ollama exists for confidential records (Section 4.4.6) but was not part of the evaluation.
- **Languages.** The assistant answers in English, Filipino, or Cebuano, depending on the question. Answers in Filipino and Cebuano were observed (Section 6.4) but not formally evaluated.
- **Public demonstration.** In the public deployment, visitors can search and chat but cannot upload documents; only administrators add to the ~~study aims to provide a clear understanding of the system's current capabilities and identify areas for improvement to effectively meet the institution's document retrieval needs.~~ library.

1.5 Significance of the Study

This study holds significant value not only for MSU-IIT students, staff, and faculty but also for other institutions and entities that require efficient document storage and retrieval systems. It aims to enhance the overall efficiency and effectiveness of managing and accessing documents within an organization.

Chapter 2 Review of Related Literature

This chapter explores the foundational principles and advancements in document retrieval systems. It examines the traditional title and keyword-based search methods alongside the more sophisticated content-based approaches. A comparative analysis is provided, highlighting the strengths, limitations, and applications of these systems in addressing the evolving demands of users. By delving into the role of OCR, vector embeddings, and LLMs, this review sets the stage for evaluating how content-based systems redefine document retrieval processes in terms of accuracy, efficiency, and user satisfaction.

2.1 Document and Information Retrieval Systems

In the modern era of digital transformation, organizations generate and store vast volumes of documents daily. This exponential growth in document storage has created an urgent need for efficient document retrieval systems (DRS) to access, manage, and utilize information effectively. A document retrieval system is a specialized application of information retrieval (IR) focused on locating relevant documents from a repository based on user queries. It plays a crucial role in knowledge management, decision-making, and enhancing productivity across various domains, including education, business, healthcare, and research.

Document retrieval is the problem of finding stored documents that contain useful information. According to [Blair et al., \(n.d.\)](#), [Blair and Maron \(1985\)](#), there exist a set of documents on a range of topics, written by different authors, at different times, and at varying levels of depth, detail, clarity, and precision, and a set of individuals who, at different times and for different reasons, search for recorded information that may be contained in some of the documents in this set. In each instance in which an individual seeks information, he or she will find some documents of the set useful and other documents not useful; the documents found useful are, we say, relevant; the others, not relevant. In order for a person to find all and only the relevant items in a collection of stored documents; one answer is automatic full-text retrieval, which on its surface is disarmingly simple: Store the full text of all documents in the collection on a computer so that every character of every word in every sentence of every document can be

located by the machine. Then, when a person wants information from that stored collection, the computer is instructed to search for all documents containing certain specified words and word combinations, which the user has specified (MacDonald & Tonello, 2020).

2.1.1 COIL: Revisit Exact Lexical Match in Information Retrieval with Contextualized Inverted List

In the realm of information retrieval, traditional systems like BM25 have long relied on exact lexical matches, efficiently utilizing inverted list indexes to retrieve documents. However, these systems often struggle with vocabulary mismatches and fail to capture the semantic nuances of language. Conversely, recent neural information retrieval (IR) models emphasize soft semantic matching across all query-document terms, enhancing semantic understanding but at the cost of computational efficiency. Addressing this dichotomy, Gao, Dai, and Callan (2021) introduced the Contextualized Inverted List (COIL) architecture, which aims to integrate the efficiency of exact lexical matching with the semantic depth provided by deep language models.

COIL innovatively stores contextualized token representations within inverted lists, enabling the system to perform exact matches based on the contextual meaning of terms rather than solely on their surface forms. This approach allows COIL to leverage the rich semantic information captured by deep language models while maintaining the retrieval speed characteristic of traditional inverted index structures. By focusing on overlapping query-document tokens' contextualized representations, COIL effectively bridges the gap between lexical and semantic matching. Empirical evaluations demonstrate that COIL outperforms both classical lexical retrievers and state-of-the-art deep language model retrievers, achieving superior retrieval effectiveness with comparable or reduced latency. This performance underscores COIL's capability to deliver precise and contextually relevant results without compromising on efficiency.

The introduction of COIL represents a significant advancement in document and information retrieval systems. By harmonizing the strengths of exact lexical matching and semantic understanding, COIL addresses longstanding challenges such as vocabulary mismatch and semantic ambiguity. Its design offers a practical solution for developing retrieval systems that are both effective and efficient, making it a valuable reference for researchers and practitioners

tinuous advancement of healthcare quality.

2.3 Information Extraction and Retrieval from PDF Documents

The Portable Document Format (PDF) is a widely adopted standard for disseminating digital documents across various domains, including academia, business, and government. Its design ensures consistent presentation across different platforms and devices, encapsulating text, images, and complex layouts. However, this fixed-layout nature poses challenges for information extraction and retrieval, as the format prioritizes visual fidelity over structural representation.

The significance of information extraction and retrieval from PDF documents lies in its ability to unlock valuable data embedded within these widely used files. By facilitating access to information that might otherwise remain obscured, effective PDF data extraction enhances data accessibility (Livathinos et al., 2021). Transforming unstructured PDF content into structured data further improves search capabilities, allowing users to efficiently locate specific information. This process also supports better knowledge management within organizations, enabling them to manage and utilize extracted information for informed decision-making and streamlined workflows (Clark et al., n.d. & Divvala, 2015).

The evolution of information extraction and retrieval from PDF documents reflects significant technological advancements. Initially, information was manually extracted from PDFs, a labor-intensive and error-prone process (Clark et al., n.d. & Divvala, 2015). This was followed by the introduction of rule-based systems, which used predefined rules and templates to identify and extract data. While these systems provided some automation, they struggled with diverse document layouts and lacked scalability. The emergence of machine learning marked a transformative phase, with models capable of learning from annotated data, thereby improving extraction accuracy and adaptability to various document structures (Palm et al., 2017). More recently, deep learning techniques have revolutionized the field by leveraging neural networks to capture complex patterns and relationships within documents. These advancements have significantly enhanced the performance of information extraction systems, enabling more effective processing of PDF content (Gupta et al., 2018).

chatbot framework powered by LangChain and Large Language Models (LLMs). Sahaay aims to revolutionize customer service by providing real-time, personalized, and context-aware support.

The development of Sahaay involves several key components. First, data collection is achieved through web scraping techniques to gather comprehensive and up-to-date information from organizational websites, ensuring the chatbot has a robust knowledge base. Second, embeddings and language models play a pivotal role, with textual data represented using embeddings and knowledge retrieval facilitated by Google's Flan T5 models (XXL, Base, and Small). These models enable Sahaay to generate accurate and contextually relevant responses. Finally, integration with customer service platforms ensures seamless embedding of the chatbot into existing support infrastructures, allowing for real-time query resolution and enhanced user interaction. This multi-faceted approach equips Sahaay to handle diverse queries effectively while aligning with organizational needs.

According to Pandya, SK., & Holia, RM. (2023), the implementation of Sahaay within an educational institution demonstrated its capability to handle diverse queries from prospective and current students, as well as researchers. The chatbot effectively provided information on topics such as notice board updates and potential research guides, showcasing its versatility and responsiveness.

This study contributes to the field of document and information retrieval systems by illustrating how integrating LLMs with open-source tools like LangChain can revolutionize customer service. By moving beyond static FAQs to dynamic, personalized interactions, organizations can enhance customer satisfaction and engagement. The research underscores the potential for scalable solutions across various industries, paving the way for more responsive and efficient customer service ecosystems.

2.7.2 Generating Breast Ultrasound Reports Using LangChain for Standardized Medical Documentation

Medical imaging plays a critical role in the early detection and diagnosis of various conditions, including breast cancer. Breast ultrasound (BUS), as a non-invasive imaging technique, is widely used for its ability to identify abnormalities in breast tissue. However, generating de-

Balancing precision and recall remains a critical concern, especially when integrating neural embeddings and LLMs

2.9 Future Trends in AI-Powered Document Retrieval

Future trends point towards deeper integration of LLMs and embeddings, development of more robust hybrid models, and enhanced capabilities in zero-shot and few-shot learning scenarios. Emphasis on ethical considerations and transparency in AI-powered retrieval systems is also expected to grow.

2.10 Summary of Related Literature

Table 2.1 maps each reviewed study to the seven themes of this chapter. Most studies address one or two themes: retrieval models such as COIL, RepBERT and CLEAR improve ranking but do not read scanned documents; LangChain applications build conversational interfaces but rely on clean text; and hybrid and retrieval-augmented approaches combine lexical and semantic signals but are evaluated on public benchmarks rather than institutional records. CatSight AI draws on all seven: it extracts text from scanned PDFs, indexes it for both semantic and keyword search, fuses the two result lists, and answers questions through an LLM agent that cites the pages it used.

Chapter 3 Theoretical Framework

This chapter lays the conceptual foundation for the development of ~~CatSight~~-AI CatSight AI. The study integrates theories and models from the domains of information retrieval (IR), ~~human-computer interaction (HCI), and~~ large language models (LLMs), ensuring technology acceptance, and human-computer interaction (HCI), so that the system addresses both technical effectiveness and user satisfaction. Section 3.1 covers the retrieval theory behind the search engine, Section 3.2 the language-model concepts behind cataloguing and chat, Section 3.3 the measures used to evaluate the system, and Sections 3.4–3.6 the theories of adoption and usability and how all of these combine in the study’s conceptual framework (Figure 3.1).

3.1 Information Retrieval Theory

Information Retrieval (IR) Theory is the study of methods and systems for finding relevant information from large collections of data, especially unstructured data such as text. It forms the foundation of technologies like search engines, digital libraries, and question-answering systems. At its core, IR involves documents—units of data stored in the system—and queries, which represent users’ information needs. To enable efficient retrieval, documents undergo preprocessing through indexing, which includes tokenization, stopword removal, stemming or lemmatization, and the construction of inverted indexes (Manning et al., 2008).

3.1.1 Tokenization

Tokenization in Information Retrieval (IR) is the process of breaking down text into smaller units called tokens, typically words or meaningful terms, which serve as the basic building blocks for indexing and matching documents to queries. It is a crucial preprocessing step that standardizes text by removing punctuation, handling case sensitivity, and separating compound words, thereby enabling efficient and accurate information retrieval. For instance, the sentence “Information retrieval is powerful” would be tokenized into [“Information”, “retrieval”, “is”, “powerful”], allowing the IR system to analyze and compare textual content effectively. The complexity of tokenization can vary depending on the

language—English tokenization is relatively straightforward, whereas languages like Chinese or Japanese require more sophisticated segmentation techniques. Tokenization facilitates the construction of structures like inverted indexes and significantly impacts retrieval performance ~~For a comprehensive discussion of tokenization, Jurafsky and Martin (2023).~~ (Jurafsky and Martin, 2023).

CatSight AI uses two kinds of tokens, and the distinction matters for the rest of this paper.

1. **Search tokens (lexemes).** For keyword search, PostgreSQL’s english text-search configuration splits a passage into words, lower-cases them, drops stop words such as *of* and *the*, and reduces each remaining word to its stem (The PostgreSQL Global Development Group, 2024). The title “Designation of the Director” becomes the lexemes *design* (position 1) and *director* (position 4). Because a query is reduced in the same way, “designated directors” still matches it. Before this step CatSight AI also strips leading zeros from numbers and splits on hyphens, so the reference “SO 01592-2023” is indexed and searched as *so*, *1592* and *2023*.
2. **Model tokens (subwords).** Language models read and write text as subword tokens produced by byte-pair encoding or a similar algorithm (Sennrich et al., 2016; Kudo and Richardson, 2018). Frequent words are single tokens, while rare words and names are split into pieces: a surname such as “Vilela-Malabanan” may take several tokens. In English text a token averages roughly three to four characters. Model limits, prices, and response times are all counted in these tokens: the *context window* is the number of tokens a model can read at once, which is why long documents need the map-reduce procedure of Section 3.2.4.

3.1.2 Vector Space Model and Embeddings

The Vector Space Model represents documents and queries as vectors in a shared space, so that relevance becomes a geometric question (Salton et al., 1975). Classical systems build these vectors from term weights; modern systems use *dense embeddings*, vectors of a few hundred to a few thousand real numbers produced by a neural encoder, in which texts with similar meaning lie close together even when they share no words (Mikolov et al., 2013; Devlin et al., 2019).

CatSight AI encodes every passage and every query with BGE-M3, a multilingual encoder that outputs vectors of $d = 1024$ dimensions (Chen et al., 2024).

3.1.3 Cosine Similarity in Semantic Search

Cosine similarity is a method used in semantic search to measure how similar two texts are by comparing their vector representations. In CatSight AI, it helps match CatSight AI, it matches the meaning of user queries with institutional documents using vectors generated from language models. This method focuses on the angle between vectors, not their length, allowing for more accurate, context-aware search results than traditional keyword matching (Huang, 2008) (Huang, 2008). Equation 3.1 defines the cosine similarity of a query vector $\mathbf{q}$ and a passage vector $\mathbf{p}$, and Equation 3.2 writes it out over the d dimensions:

$$\cos(\mathbf{q}, \mathbf{p}) = \frac{\mathbf{q} \cdot \mathbf{p}}{\|\mathbf{q}\| \|\mathbf{p}\|} \quad (3.1)$$

$$= \frac{\sum_{i=1}^d q_i p_i}{\sqrt{\sum_{i=1}^d q_i^2} \sqrt{\sum_{i=1}^d p_i^2}} \quad (3.2)$$

where $\mathbf{q} \cdot \mathbf{p}$ is the dot product of the two vectors, $\|\mathbf{q}\|$ and $\|\mathbf{p}\|$ are their magnitudes (Euclidean norms), and q_i and p_i are their i -th components.

Interpreting the score. The cosine similarity ranges from -1 (opposite directions) through 0 (unrelated, orthogonal vectors) to 1 (the same direction). Because only the angle matters, a long passage and a short query can still score highly. The pgvector extension used by CatSight AI computes the *cosine distance* of Equation 3.3, and the search orders passages by increasing distance, which is the same as decreasing similarity (Kane, 2024):

$$\text{dist}(\mathbf{q}, \mathbf{p}) = 1 - \cos(\mathbf{q}, \mathbf{p}) \quad (3.3)$$

With BGE-M3 on this corpus, passages that answer a question typically score between 0.55 and 0.75, while unrelated passages score between 0.30 and 0.40. CatSight AI therefore treats a passage found only by vector search as relevant when $\cos(\mathbf{q}, \mathbf{p}) \geq \tau$ with $\tau = 0.42$. **Equation 1,** shows the equation of getting the cosine similarity of the semantic search.

`\begin{equation}`

`\text{Cosine Similarity} = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|}`

`\end{equation}`

Explanation:

`\vec{A}` and `\vec{B}` denote vectors A and B

–

`\cdot` represents the dot product

–

`\|\vec{A}\|` and `\|\vec{B}\|` are the magnitudes (norms) of the vectors

Worked example. Consider three-dimensional vectors (real embeddings have 1024 dimensions) for a query $\mathbf{q} = (0.8, 0.1, 0.6)$, a related passage $\mathbf{a} = (0.7, 0.2, 0.7)$, and an unrelated passage $\mathbf{b} = (0.1, 0.9, 0.2)$. Then $\|\mathbf{q}\| = \sqrt{1.01} \approx 1.005$, $\|\mathbf{a}\| = \sqrt{1.02} \approx 1.010$ and $\|\mathbf{b}\| = \sqrt{0.86} \approx 0.927$, so

$$\cos(\mathbf{q}, \mathbf{a}) = \frac{0.56 + 0.02 + 0.42}{1.005 \times 1.010} \approx 0.985, \quad \cos(\mathbf{q}, \mathbf{b}) = \frac{0.08 + 0.09 + 0.12}{1.005 \times 0.927} \approx 0.311.$$

Passage $\mathbf{a}$ points in nearly the same direction as the query and is ranked first; passage $\mathbf{b}$ falls below the threshold $\tau = 0.42$ and is discarded unless keyword search also finds it.

3.1.4 Lexical Ranking

Dense vectors capture meaning but blur exact identifiers: two special orders with different numbers look almost identical to an embedding model. Lexical retrieval ranks passages by the query terms they contain, as in the classic BM25 family of models (Robertson and Zaragoza, 2009). CatSight AI uses PostgreSQL full-text search for this: each passage stores a `tsvector` of its lexemes, in which the passage’s context header (title, reference number, year) is weighted higher than its body, and passages are ranked with the cover-density function `ts_rank_cd`, which rewards passages where the query terms occur close together (The PostgreSQL Global Development Group, 2024). The query is the logical OR of its terms, so a passage does not need to contain every word of a question to be found.

3.1.5 Reciprocal Rank Fusion

The vector and keyword searches return two ranked lists whose scores are not comparable: a cosine similarity of 0.6 and a `ts_rank_cd` of 0.1 are on different scales. Reciprocal Rank Fusion (RRF) combines lists by rank alone (Cormack et al., 2009). For a passage p and a set of rankings R ,

$$\text{RRF}(p) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(p)} \quad (3.4)$$

where $\text{rank}_r(p)$ is the 1-based position of p in ranking r (a ranking that does not contain p contributes nothing) and k is a damping constant, set to the customary $k = 60$. A passage ranked first by keyword search and fourth by vector search scores $\frac{1}{61} + \frac{1}{64} \approx 0.0320$, ahead of a passage ranked second by vector search only, at $\frac{1}{62} \approx 0.0161$. Agreement between the two searches is thus rewarded without either one having to be calibrated against the other (Bruch et al., 2022).

3.2 Large Language Models in IR

Recent advancements in transformer-based LLMs such as BERT (Devlin et al., 2018) and GPT (Radford et al., 2019) (Devlin et al., 2019) and GPT (Radford et al., 2019; Brown et al., 2020) have revolutionized retrieval-based tasks through semantic search and contextual query understanding. CatSight AI-CatSight AI adopts such LLMs to handle open-ended queries with improved precision, echoing approaches validated in contemporary research (Gao et al., 2021). (Gao et al., 2021; Zhu et al., 2023). A transformer reads a sequence of tokens and, through layers of self-attention, predicts a probability distribution over the next token (Vaswani et al., 2017); generation repeats this prediction one token at a time (Figure 4.2 in Chapter 4).

LLM Hallucinations

3.2.1 Zero-Shot Prompting

A model can be adapted to a task in three ways. *Fine-tuning* updates the model's weights on labelled examples of the task. *Few-shot prompting* leaves the weights unchanged but includes a handful of solved examples in the prompt. *Zero-shot prompting* gives only a natural-language description of the task, with no examples at all (Brown et al., 2020). Instruction-tuned models

follow such descriptions well (Wei et al., 2022), and zero-shot prompting can even elicit multi-step reasoning (Kojima et al., 2022).

CatSight AI is zero-shot throughout. No model was fine-tuned and no prompt contains worked examples of the task: the transcription of each scanned page, the cataloging of each document (title, summary, reference number, date, tags and suggested questions), the chat answers, and the chat titles are all produced from written instructions alone. Two techniques make zero-shot output dependable. First, cataloging uses *structured output*: the model must return JSON that matches a fixed schema, so every field is present and typed. Second, the model's choices are constrained by data the system controls—for example, tags must be copied from the list of existing tags and their descriptions, and anything else is discarded. Zero-shot prompting keeps the system adaptable: adding a document type or a tag needs a new description, not new training data.

3.2.2 LLM Hallucination

Hallucination in Large Language Models (LLMs) refers to the generation of text that is fluent and contextually relevant but factually incorrect or fabricated. This phenomenon poses a significant challenge in systems like ~~CatSight AI~~CatSight AI, which rely on LLMs to interpret and retrieve information from documents. Since LLMs are trained on vast, mixed-quality datasets using statistical language modeling, they generate responses based on learned word patterns rather than verified facts. Without grounding in external knowledge bases or the original document context, LLMs may produce answers that appear relevant but do not accurately reflect the source material. This issue is particularly critical in document retrieval systems, where factual accuracy and source traceability are essential. Furthermore, reinforcement techniques like RLHF (Reinforcement Learning from Human Feedback) may unintentionally favor responses that are convincing over those that are correct. Understanding and mitigating hallucination is thus crucial to improving the reliability and trustworthiness of ~~CatSight AI~~CatSight AI's LLM-based chat interface (Ji et al., 2023)(Ji et al., 2023).

Types. Ji et al. (2023) distinguish *intrinsic* hallucinations, which contradict the source the model was given (for example, stating the wrong effectivity date of a special order that is in

the context), from *extrinsic* hallucinations, which cannot be verified from the source at all (for example, naming an official who appears in none of the retrieved passages).

Basis. The causes lie in both data and modelling. Training text contains errors and statements that diverge from their sources; the model stores knowledge implicitly in its parameters and may prefer this *parametric* knowledge over the passage in front of it; it is trained to predict the most likely next token, not the true one; and sampling during decoding adds randomness, so a fluent but unsupported continuation can be chosen (Ji et al., 2023). Questions about a specific institution’s records are especially exposed, because the model has seen little or none of that information in training.

Mitigation in CatSight AI. The system applies retrieval-augmented generation (Section 3.2.3) and four further measures, described in detail in Section 5.3.10: (1) the assistant is instructed to answer only from the retrieved passages and to say so plainly when they do not contain the answer; (2) every claim must carry a numbered citation to a passage, and the user can open the cited page; (3) answers are generated at a low temperature (0.2), which reduces sampling randomness; and (4) questions unrelated to the university’s documents are declined.

3.2.3 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) conditions a language model on passages retrieved for the input, instead of relying only on what the model memorized (Lewis et al., 2020). In its probabilistic form, the probability of an answer y to a question x marginalizes over the top- k retrieved passages z :

$$p(y | x) \approx \sum_{z \in \text{top-}k(p_\eta(\cdot|x))} p_\eta(z | x) p_\theta(y | x, z) \quad (3.5)$$

where p_η is the retriever and p_θ the generator. CatSight AI uses the common practical simplification: the retriever is the hybrid search of Section 3.1.5, and the k passages it returns are placed together in the model’s context, numbered so that the answer can cite them. Because the model can call the search tool several times per question (Section 4.6.4), it can also reformulate a search when the first results are insufficient, in the spirit of adaptive retrieval (Asai et al., 2023;

Jeong et al., 2024).

3.2.4 Map-Reduce Summarization

MapReduce is a programming model for processing data too large for one machine: a *map* function is applied independently to each piece of the input, and a *reduce* function combines the intermediate results (Dean and Ghemawat, 2004). The same idea applies to text that is too long for one model call. Let a document's text T of length n characters exceed the budget B that one call may read. T is split into sections $c_1, \dots, c_m$ of at most S characters each (with a small overlap), and each section is summarized independently, in parallel:

$$T = c_1 \oplus c_2 \oplus \dots \oplus c_m, \quad |c_i| \leq S, \quad n_i = f_{\text{map}}(c_i) \quad (3.6)$$

where $\oplus$ denotes concatenation and n_i are compact notes that keep names, dates, amounts and reference numbers. If the notes together still exceed the budget, the map step is applied again to the notes themselves (a reduce round), at most three times. The final structured call then reads the document's opening $T_{[1:O]}$, which holds its letterhead, subject and reference number, together with the notes:

$$A = f_{\text{final}}(T_{[1:O]}, n_1, \dots, n_m) \quad \text{when} \quad \sum_{i=1}^m |n_i| \leq B - O \quad (3.7)$$

In CatSight AI, $B = 300,000$, $S = 60,000$ and $O = 4,000$ characters for hosted models. Because most administrative issuances are a few pages long, nearly every document is catalogued in a single call ($n \leq B$), and map-reduce is the fallback for long resolutions and policies. With a local model limited to 8,192 tokens, the budgets shrink to $B = 14,000$ and $S = 6,000$ characters, and map-reduce becomes the common path. The procedure is given as Algorithm 3 and drawn in Figure 4.5.

3.3 System Evaluation

CatSight AI is evaluated on four qualities: whether it retrieves the right documents, whether its summaries are faithful, whether it responds quickly, and whether people find it usable. This section defines the measure used for each; Section 4.7 describes how they are applied.

3.3.1 Retrieval Accuracy

For a test question q with a set of relevant documents Rel_q , let $Ret_{q,k}$ be the first k documents the system returns. Precision and recall at k (Manning et al., 2008), and the mean reciprocal rank over a set of questions Q (Voorhees, 1999), are

$$P@k = \frac{|Rel_q \cap Ret_{q,k}|}{k}, \quad R@k = \frac{|Rel_q \cap Ret_{q,k}|}{|Rel_q|}, \quad (3.8)$$

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q} \quad (3.9)$$

where rank_q is the position of the first relevant document for question q (the term is zero if none is returned).

3.3.2 Summary Quality: ROUGE, BLEU and BERTScore

Each document's generated summary $\hat{y}$ is compared with a reference summary y written by a person (Section 4.7). Three complementary metrics are used.

ROUGE. ROUGE-N measures the recall of reference n-grams in the generated summary (Lin, 2004):

$$\text{ROUGE-N} = \frac{\sum_{g \in \text{grams}_N(y)} \text{Count}_{\text{match}}(g)}{\sum_{g \in \text{grams}_N(\hat{y})} \text{Count}(g)} \quad (3.10)$$

where $\text{grams}_N(y)$ are the n-grams of length N in the reference and $\text{Count}_{\text{match}}(g)$ is the number of times g also occurs in $\hat{y}$ (at most its count in $\hat{y}$). ROUGE-1 and ROUGE-2 use unigrams and bigrams. ROUGE-L uses the longest common subsequence (LCS) of the reference (m words) and the candidate (n words), which rewards in-order matches without requiring them to be contiguous:

$$R_{\text{lcs}} = \frac{\text{LCS}(y, \hat{y})}{m}, \quad P_{\text{lcs}} = \frac{\text{LCS}(y, \hat{y})}{n}, \quad F_{\text{lcs}} = \frac{(1 + \beta^2) R_{\text{lcs}} P_{\text{lcs}}}{R_{\text{lcs}} + \beta^2 P_{\text{lcs}}} \quad (3.11)$$

with $\beta = 1$ weighting precision and recall equally.

BLEU. BLEU measures the precision of the candidate's n-grams against the reference, with counts clipped so that repeating a correct word is not rewarded, and a brevity penalty that stops

very short candidates from scoring well (Papineni et al., 2002):

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right), \quad \text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{1-r/c} & \text{if } c \leq r \end{cases} \quad (3.12)$$

where p_n is the clipped n -gram precision, $N = 4$ with uniform weights $w_n = \frac{1}{4}$, c is the length of the candidate and r the length of the reference.

BERTScore. ROUGE and BLEU only credit identical words, so a correct paraphrase (“designates” for “appoints”) scores poorly. BERTScore instead matches each token to its most similar token in the other text using contextual embeddings and cosine similarity (Equation 3.1) (Zhang et al., 2020). With $\mathbf{x}_i$ the normalized embeddings of the reference tokens and $\hat{\mathbf{x}}_j$ those of the candidate,

$$R_{\text{BERT}} = \frac{1}{|y|} \sum_{\mathbf{x}_i \in y} \max_{\hat{\mathbf{x}}_j \in \hat{y}} \mathbf{x}_i^\top \hat{\mathbf{x}}_j, \quad P_{\text{BERT}} = \frac{1}{|\hat{y}|} \sum_{\hat{\mathbf{x}}_j \in \hat{y}} \max_{\mathbf{x}_i \in y} \mathbf{x}_i^\top \hat{\mathbf{x}}_j, \quad F_{\text{BERT}} = \frac{2 P_{\text{BERT}} R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \quad (3.13)$$

Together, the three metrics show whether a summary keeps the reference’s key terms (ROUGE), avoids adding unsupported wording (BLEU), and preserves its meaning even when phrased differently (BERTScore).

3.3.3 Efficiency

Responsiveness is measured as the *time to first token*, from sending a question to the first word of the answer appearing, and the *total response time*, until the answer is complete. Because answers stream, the first measure governs how responsive the system feels. For search, the server reports the time taken to retrieve and rank passages. For ingestion, throughput is measured in pages processed per minute.

3.3.4 Usability: System Usability Scale

The System Usability Scale (SUS) is a ten-item questionnaire answered on a five-point scale (Brooke, 1996). Odd-numbered items are positively worded and even-numbered items nega-

tively worded, so the score $s_i \in \{1, \dots, 5\}$ of item i contributes as follows:

$$\text{SUS} = 2.5 \left[\sum_{i \in \{1,3,5,7,9\}} (s_i - 1) + \sum_{i \in \{2,4,6,8,10\}} (5 - s_i) \right] \quad (3.14)$$

giving a score from 0 to 100. Scores are commonly interpreted against an average of about 68; scores in the 70s correspond to “good” usability and scores around 50 to “OK” or marginal usability (Bangor et al., 2009).

3.4 Technology Acceptance Model

The Technology Acceptance Model (TAM) explains whether people adopt a new system through two beliefs: *perceived usefulness*, the degree to which a person believes the system would improve their work, and *perceived ease of use*, the degree to which they believe using it would be free of effort (Davis, 1989). Both shape the attitude toward the system and, through it, the intention to use it. For CatSight AI, perceived usefulness rests on retrieving the right issuance and answering with verifiable citations, which the retrieval and summary measures of Section 3.3 assess. Perceived ease of use rests on asking in one’s own words instead of guessing the keywords a document uses, which the SUS assesses. The model explains a design decision of this study: an accurate system that is hard to use, or an easy one whose answers cannot be trusted, would not be adopted by MSU-IIT’s students, faculty, and staff.

3.5 Cognitive Load Theory and Usability

Drawing from Cognitive Load Theory (Sweller, 1988), CatSight AI (Sweller, 1988), CatSight AI is designed to minimize user confusion through a clean interface and structured interactions. This theory guides UI/UX choices, ensuring users can process and retrieve information without cognitive overload (Paas & Van Merriënboer, 1994) (Paas and Van Merriënboer, 1994). Concretely, answers lead with a one- or two-sentence direct answer before details; citations are short numbered markers that open the source passage on demand rather than long quotations; suggested questions remove the effort of composing a first query; and the interface uses one consistent layout for the dashboard, documents, search, and chat (Section 4.6.5).

Summary of Theoretical Model

3.6 Conceptual Framework

The integrated framework—composed of IR principles, LLM-driven search, TAM for adoption, and cognitive load for usability—guides both the functional and experiential design of CatSight AI. Figure 3.1 presents it as an input–process–output model. The *inputs* are the scanned and digital issuances, the users’ questions, and the tag definitions and limits set by administrators. The *process* applies the theories of this chapter in sequence: OCR turns page images into text; zero-shot cataloguing with map-reduce describes each document; tokenization and embeddings index each passage for both keyword and vector search; cosine similarity, lexical ranking, rank fusion, and reranking retrieve the passages for a question; and retrieval-augmented generation answers from them with citations, limiting hallucination. The *outputs* are a searchable library, cited answers, and the evaluation results that test the framework: retrieval accuracy, ROUGE, BLEU and BERTScore for summaries, response times, and SUS scores. This synthesis ensures a technically sound, user-friendly system tailored to MSU-IIT’s academic needs.

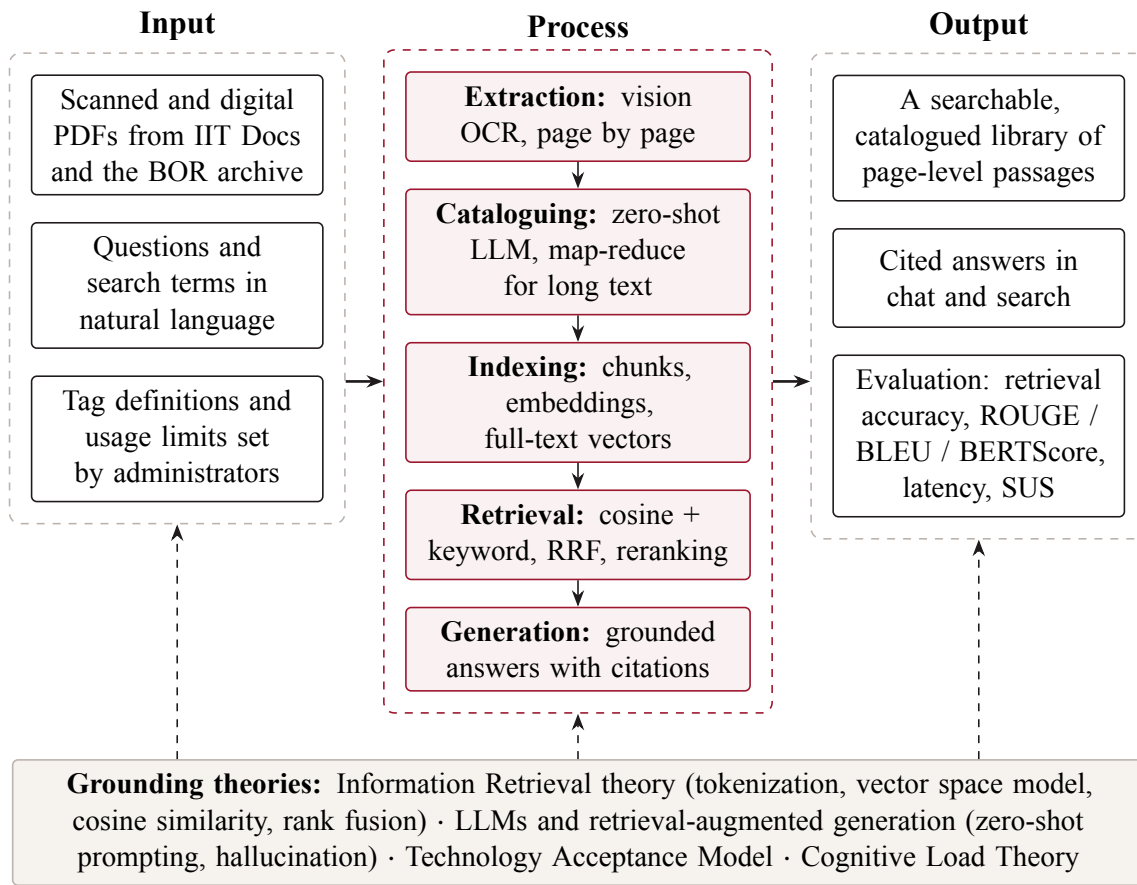
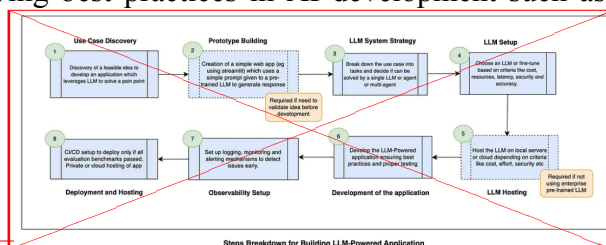


Figure 3.1. *Conceptual Framework of the Study*

Chapter 4 Research Methodology

This chapter outlines the methodology used to develop ~~CatSight AI~~**CatSight AI**, a document retrieval system (DRS) tailored for Mindanao State University – Iligan Institute of Technology (MSU-IIT). ~~Adapted~~It is adapted from Pallavi Sinha’s framework for building LLM-powered applications (Sinha, 2024), shown in Figure 4.1. The framework has eight steps; this study follows the first six, ~~the process excludes deployment-related steps to align with the project’s timeline (January–April 2025) as shown in **Figure 2**.~~ However, this study only follows six key phases. ~~The structured approach ensures that CatSight AI meets MSU-IIT’s specific needs, following best practices in AI development such as iterative testing and data-driven~~



refinement.



Steps Breakdown for Building LLM-Powered Application

Steps Breakdown for Building

LLM-Powered Applications (Sinha, 2024) Only six key phases were adopted to ensure the feasibility and relevance of the system during its limited from use case discovery to the development of the application, and leaves out observability and deployment pipelines, which fall outside the project's development window. This—The structured approach ensures that CatSight AI—CatSight AI meets MSU-IIT's specific needs, following best practices in AI development ,—such as iterative testing, ethical considerations, and data-driven refinement (Amershi et al., 2019)(Amershi et al., 2019). Sections 4.1 to 4.6 follow the six adopted phases in order, and Section 4.7 describes how the system is evaluated.

4.1 Use Case Discovery

The proponents observed that MSU-IIT’s current document retrieval system posed challenges for students, faculty, and staff. The keyword-based search often failed to return relevant results when queries didn’t match exact terms, making it difficult to find necessary documents. For instance, searching for “policy updates” might miss files labeled differently, such as “revised regulations.” These limitations revealed the system’s lack of contextual understanding

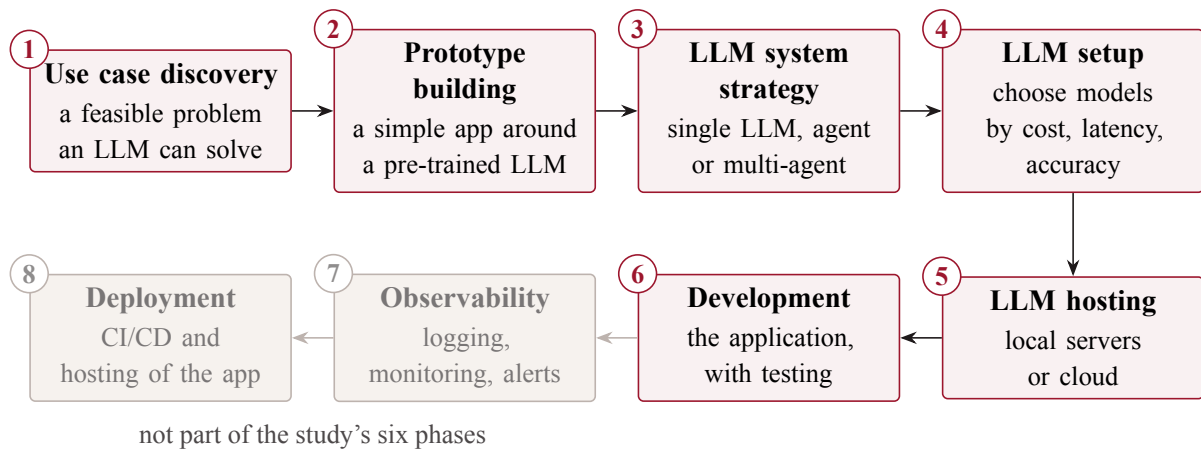


Figure 4.1. *Steps for Building LLM-Powered Applications, Adapted from Sinha (2024); Steps 1–6 Were Adopted*

and highlighted the need for a more intelligent solution. ~~CatSight AI~~ CatSight AI addresses this by integrating OCR, vector similarity search, and LLM-based chat interfaces, enabling semantic search and conversational interaction to improve search accuracy and user experience.

4.1.1 Process of Document Collection and Preparation

Building on the identified limitations of MSU-IIT’s traditional document retrieval system, the study moves toward developing and evaluating a more intelligent solution—~~CatSight AI~~ solution. To ensure that the system is tested under realistic and representative conditions, the process begins with the careful collection and preparation of institutional documents. These documents are sourced from two main repositories: (1) MSU-IIT’s internal document archives archive, IIT Docs (Mindanao State University - Iligan Institute of Technology, ndb), and (2) the MSU System Board of Regents (BOR) Resolutions archive (Mindanao State University - Iligan Institute of Technology, nda).

Both scanned and text-based PDF files are gathered, curated, and pre-processed to serve as the dataset for evaluating the proposed ~~Document Retrieval System (DRS)~~ DRS. This step ensures the inclusion of a wide variety of document types, formats, and complexities, particularly those that require Optical Character Recognition (OCR) for ~~semantic interpretation.~~

interpretation. To maintain relevance and comprehensiveness, specific criteria were observed in selecting the documents:

- **Relevance:** ~~÷ Files~~ files were chosen based on their alignment with commonly refer-

enced administrative functions and institutional topics.

- **Diversity of Content**~~content~~: ~~÷ The dataset includes various categories such as policies, appointments, and event-related~~ the dataset includes policies, designations, incentives, travel orders, suspensions, and Charter Day issuances.
- **Format Variety**~~variety~~: ~~÷ Both both~~ scanned and digitally native-produced PDFs were included to assess the system's performance in handling handling of OCR and different file structures.
- **Structural Complexity**~~complexity~~: ~~÷ Documents featuring documents with~~ tables, multiple sections, and complex layouts were prioritized to rigorously test the DRS's retrieval capabilities watermarks, and typewritten text were included to test the retrieval capabilities rigorously.

Table 4.1 summarizes the resulting corpus of 50 PDF files: five per category and repository, 25 from each repository. Two files are byte-for-byte duplicates of other files and are recognized as such on upload, leaving 48 distinct documents with 158 pages, issued between 1970 and 2024. Thirteen files have no text layer at all (pure scans), and most of the others carry a text layer produced by the repositories' own OCR, of varying quality; 33 of the 48 documents are a single page long, and the longest has 36 pages.

Table 4.1. *Composition of the Document Corpus*

Category	IIT Docs	BOR Resolutions	Total
Charter Day	5	—	5
Designation	5	5	10
Incentive	5	5	10
Policy	5	5	10
Suspension	—	5	5
Travel Order	5	5	10
Total	25	25	50

This comprehensive document preparation process sets the foundation for a meaningful evaluation of ~~CatSight AI~~CatSight AI's ability to overcome the limitations of the current system and deliver more accurate, context-aware search results.

Problem Definition & Stakeholders

4.1.2 Problem Definition and Stakeholders

The development of ~~CatSight AI~~ CatSight AI was motivated by critical limitations observed in the existing ~~Document Retrieval System (DRS) at Mindanao State University – Iigan Institute of Technology (DRS at MSU-IIT)~~. The institution relies on two primary repositories: the ~~Board of Regents (BOR)~~ BOR Resolutions Archive, which governs MSU System policies, and IIT Docs, a repository of Special Orders, Memorandum Orders, and administrative issuances built on Google Drive's keyword-based search. While functional, these systems exhibit significant shortcomings:

1. ~~Semantic Blindness:~~ **Semantic blindness.** The keyword-only mechanism fails to interpret contextual nuances or semantic relationships, leading to irrelevant or incomplete results. ~~For example:~~
2. ~~Queries for "Special Orders"~~ Queries for "Special Orders" miss documents labeled with abbreviations like "SO."
3. ~~Synonym-rich content (e.g., "academic calendar" vs. "term schedule")~~ "SO"; synonym-rich content ("academic calendar" versus "term schedule") remains undiscovered.
4. ~~Paraphrased queries (e.g., "student allowances" vs. "financial aid")~~ and paraphrased queries ("student allowances" versus "financial aid") yield inconsistent results.
5. ~~Format Exclusion:~~ **Format exclusion.** Scanned or image-based PDFs are entirely excluded from search results ~~due to the lack of Optical Character Recognition (OCR) capabilities~~ because the repositories lack OCR. Documents with complex layouts—such layouts, such as tables, multi-column text, or handwritten annotations—are annotations, are similarly unsearchable.

6. ~~Rigid Interaction:~~ **Rigid interaction.** Users cannot refine queries iteratively due to the absence of a because there is no conversational interface, forcing which forces them to rely on repetitive keyword adjustments.

The primary stakeholders are ~~Primary Stakeholders:~~ **students**

~~Students: Require,~~ who need timely access to academic policies (e.g. for example, scholarship resolutions , and enrollment guidelines) and event announcements. ~~Staff: Responsible for locating memorandums (e.g., and staff,~~ who locate memoranda (for example, procurement procedures , and compliance records) to streamline operational workflows.

4.2 Agent Prototype Building

~~LLM System Strategy~~

The first prototype was built between January and April 2025 to validate the idea before full development. It used a Django REST API, a React interface, Marker for OCR, LangChain's recursive text splitter, a pgvector store, and models running locally in Ollama. Several local models were compared on sample MSU-IIT documents for text generation, question answering, and semantic similarity search, considering response accuracy, latency, and resource use (Table 4.2).

~~Strategic design decisions were made, including model selection, embedding strategies, and chunking techniques for documents. The LLM System Strategy for CatSight AI prioritized a single LLM approach to streamline development and reduce computational overhead, opting against multi-agent systems that are better suited for complex, specialized tasks (Chen et al., 2023). This decision was informed by a rigorous evaluation of multiple candidate models from Ollama to ensure optimal performance for document retrieval and conversational interaction at MSU-IIT. Several models were tested for both the primary LLM and the embedding model:~~

~~For the LLM:~~ The prototype established three lessons that shaped the final design. First,

Table 4.2. *Local Models Compared During Prototype Building (January–April 2025)*

Model (Ollama)	Observation
<i>Language models</i>	
llama3.1:8b	Strong language understanding, but too demanding for real-time use on the available hardware.
llama3.2:1b	Light, but insufficient for conversations that need deep contextual understanding.
qwen3:1.7b	Good language ability, but struggled with long documents.
qwen2.5:7b-instruct-q4_K_M	Followed instructions well, with higher latency.
llama3.1:8b-text-fp16	Good text processing at a higher resource cost.
llama3.1:8b (4-bit)	Selected for the prototype: the best balance of quality and resource use (Grattafiori et al., 2024).
<i>Embedding models</i>	
bge-m3	Multilingual, but in this prototype weaker on names, numbers and local terms.
mxbai-embed-large	Selected for the prototype for its precision on such entities (1024 dimensions) (Mixedbread AI, 2024).

on CPU-only hardware even the selected 8-billion-parameter model answered slowly, and OCR of a scanned page took longer still, so model hosting had to be separated from the application (Section 4.5). Second, dense embeddings alone missed exact identifiers such as reference numbers and surnames, whichever embedding model was used; the final system therefore pairs BGE-M3 with a keyword index and fuses the two (Section 3.1.5), which recovers exactly the queries on which BGE-M3 had looked weaker. Third, answers needed visible evidence to be trusted, which led to numbered, page-level citations.

- ~~llama3.1:8b~~: Demonstrated strong language understanding but required significant computational resources, making it less feasible for real-time applications.
- ~~llama3.2:1b~~: A lightweight model with low resource demands but insufficient for handling complex conversational tasks requiring deep contextual understanding.
- ~~qwen3:1.7b~~: Offered good language capabilities but struggled with processing long-context documents, a critical requirement for MSU-IIT’s use case.

- ~~qwen2.5:7b-instruct-q4_K_M~~: Excelled in instruction-following tasks but was less efficient for real-time interaction due to higher latency.
- ~~llama3.1:8b-text-fp16~~: Optimized for text processing, it performed well but was less versatile than Llama 3.1 for conversational applications but requires more resources.
- ~~llama3.18b-q4~~: Selected as the primary LLM for its balanced performance, efficiency, and ability to handle complex queries and generate context-aware responses, fine-tuned for MSU-HT's institutional context (Meta AI, 2025).

4.3 LLM System Strategy

~~For the embedding model:~~ Strategic design decisions were made on model selection, embedding strategy, and document chunking. CatSight AI uses a *single agent with one tool*: one language model reads the question and the conversation, decides whether and what to search, calls the search tool up to three times per question, and writes the answer from the passages it retrieved. This was preferred over a multi-agent design, which suits complex tasks that need several specialized roles (Chen et al., 2023); answering from a document library needs one well-grounded retrieval loop, and a single agent is simpler to build, faster, and easier to evaluate. Document processing, by contrast, is a fixed pipeline rather than an agent (Section 4.6.3), because every document goes through the same steps.

- ~~bge-m3:latest~~: Initially evaluated for its multi-lingual support but found lacking in accurately capturing specific entities such as names, numbers, and native terms prevalent in MSU-HT's documents.
- ~~mxbai-embed-large~~: Chosen for its superior semantic accuracy, particularly in handling nuanced entities, making it ideal for semantic search across MSU-HT's diverse document set (mixedbread.ai, 2024).

All language models are used zero-shot (Section 3.2.1). Every model in the system is *open-*

weight: its weights are published under a license that allows the institution to download and run it on its own hardware. Because the proponents have no GPU server of their own, the models are reached through OpenRouter, a service that hosts the same open models behind one API; the system can be moved to self-hosted copies of the same weights without changing its behavior (Section 4.5). Table 4.3 lists the models and their roles.

~~Benchmarking was conducted using sample documents from MSU-IT, focusing on tasks such as text generation, question answering, and semantic similarity search. Metrics included response accuracy, latency, and resource utilization. Llama 3.1 8B Q4 outperformed other LLMs in generating accurate and contextually relevant responses with moderate computational requirements, making it suitable for real-time applications. Similarly, mxbai-embed-large demonstrated higher precision in embedding tasks compared to bge-m3, particularly for documents containing names, numbers, and native terms, ensuring robust retrieval performance (Brown et al., 2020; mixedbread.ai, 2024).~~

Table 4.3. *Open-Weight Models Used by CatSight AI*

Model	License	Role in CatSight AI
Surya OCR 2 (0.69B)	OpenRAIL	Marker’s OCR engine: reads the text of every page, run locally with llama.cpp.
Qwen3-VL-30B-A3B-Instruct	Apache-2.0	Marker’s LLM mode (tables, forms, handwriting), cataloguing, chat answers, chat titles.
BGE-M3 (0.57B)	MIT	Embeddings of passages and queries (1024 dimensions, multi-lingual).
Qwen3-Reranker-8B	Apache-2.0	Reranking of the fused search results.

4.4 LLM Setup

One of the key processes of this study is choosing the right LLM setup. This section discusses the LLM setup chosen by the proponents; explains how a language model works and why each model in Table 4.3 was chosen.

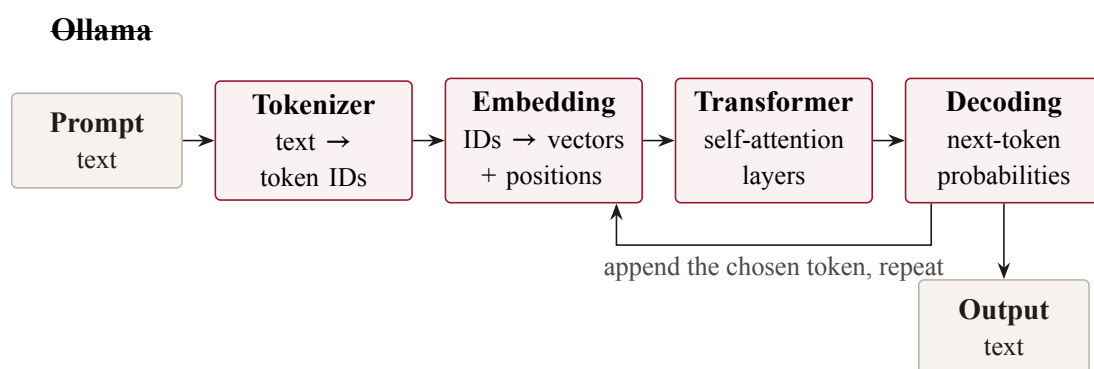
4.4.1 LLM Underlying Process

According to Mulugeta (2023) Mulugeta (2024), a Large Language Model (LLM) operates through a series of structured processes to interpret and generate human-like text. A simplified

representation of this workflow is illustrated in the following image—

IMAGE

The article then provides a deeper exploration of several key steps in the process while intentionally excluding more advanced topics—such as model training—that fall outside the scope of this study: CatSight AI uses pre-trained models as they are.



Ollama is an open-source framework optimized for local deployment and efficient inference of large language models (LLMs). In CatSight AI, Ollama serves as the foundational LLM engine, running entirely on-premises within MSU-IIT’s infrastructure to guarantee compliance with the Philippine Data Privacy Act. It integrates seamlessly with Django’s REST framework to expose inference endpoints and with our Retrieval-Augmented Generation (RAG) pipeline for context-aware academic document retrieval. This setup balances low-latency performance, domain-specific customization (e.g., adherence to MSU-IIT’s document schemas and terminologies), and institutional governance requirements—ensuring a scalable, privacy-preserving AI solution tailored to an academic environment

Figure 4.2. *How a Large Language Model Generates Text, After Mulugeta (2024)*

4.4.2 Marker and Surya OCR 2

Marker converts PDF files to Markdown (Datalab, 2025a). Version 2 performs OCR with Surya OCR 2, a 0.69-billion-parameter vision-language model trained to transcribe document images, which it serves through llama.cpp, an efficient inference engine for open models (Datalab, 2025b; Gerganov and ggml-org contributors, 2025). Marker first detects the text lines and layout blocks of each page, then has Surya OCR 2 read every block, ignoring any existing text layer, and finally assembles the blocks into Markdown in reading order. Marker was the OCR tool of the defended prototype; version 2 replaces its earlier recognition models with this vision-language model.

LLM mode. Marker can additionally send the regions it finds hardest—tables, forms, handwriting, and complex regions—to a general vision-language model, which corrects their text using the image (Datalab, 2025a). CatSight AI enables this mode with Qwen3-VL-30B-A3B-Instruct. The combination keeps the strength of each model: Surya OCR 2 transcribes characters faithfully, including unusual surnames that a general model may “normalize”, while Qwen3-VL restructures the tables of stipend rates and incentive amounts that line-by-line OCR tends to flatten. Section 6.1 compares this setup with Docling, MarkItDown, and the vision model on its own.

4.4.3 Qwen3-VL-30B-A3B-Instruct

Qwen3-VL-30B-A3B-Instruct is an open-weight vision-language model released under the Apache-2.0 license (Qwen Team, 2025b). It is a *mixture-of-experts* model: of its roughly 31 billion parameters, only about 3 billion are active for each token, so it answers about as fast as a 3-billion-parameter model while drawing on the knowledge of a much larger one. It accepts page images, follows instructions in many languages including Filipino, calls tools, and produces output that conforms to a JSON schema, which covers every language task in CatSight AI: Marker’s LLM mode, cataloguing, chat answers, and chat titles. Using one model for all four keeps self-hosting simple, since only one large model has to be served.

Llama 3.1:8b-Q4

4.4.4 BGE-M3

Llama 3.1's 8-billion-parameter variant, quantized to 4 bits per weight (Q4), offers nearly FP16-level performance ($\approx 99.3\%$ relative quality) while reducing memory footprint by over 4 $\times$ —making on-premises inference feasible on modest hardware of 4–8 GB RAM (Meta AI, 2024). This quantized model runs natively in Ollama, enabling CatSight AI to deliver fast, context-aware responses without offloading data externally. BGE-M3 is a multilingual text-embedding model of 0.57 billion parameters released under the MIT license (Chen et al., 2024). It maps passages of up to 8,192 tokens to 1024-dimensional vectors and supports over 100 languages, so questions in Filipino or Cebuano can still find English passages. It is small enough to run on a CPU.

mxbai-embed-large

4.4.5 Qwen3-Reranker-8B

The mxbai-embed-large model from Mixedbread AI produces dense, 768-dimensional sentence embeddings optimized for semantic retrieval. Trained without overlap on the MTEB benchmark, it outperforms many same-sized and even larger proprietary models (Mixedbread AI, 2024). This ensures high-quality vector representations for MSU-IIT's diverse document corpus. In CatSight AI, these embeddings power our pgvector store, underpinning precise similarity search in the RAG pipeline. A reranker, or cross-encoder, reads the query and a candidate passage *together* and outputs a relevance score (Nogueira and Cho, 2019). This is more accurate than comparing two independently computed embeddings, but too slow to apply to the whole library, so CatSight AI applies it only to the 20 best candidates after rank fusion. Qwen3-Reranker-8B is released under the Apache-2.0 license (Qwen Team, 2025a).

4.4.6 Local Configuration with Ollama

For confidential records that must not leave the campus network, as required by the Data Privacy Act of 2012 (Republic of the Philippines, 2012), CatSight AI can run every model locally

through Ollama, an open-source runtime for language models (Ollama, 2024). In this configuration the answers and catalogue come from Qwen3-4B-Instruct (4-bit), chat titles from Qwen3-1.7B, embeddings from BGE-M3, OCR from Marker with Surya OCR 2, and Marker's LLM mode from a local vision model such as Qwen3-VL-8B. The switch is a single setting; the pipeline, search, and agent are unchanged. Because the smaller local models have an 8,192-token context window, long documents take the map-reduce path of Algorithm 3 (Section 3.2.4).

4.5 LLM Hosting

~~CatSight AI's core LLM is hosted~~ CatSight AI separates the application from the models. The application—the web interface, the API, the document pipeline, and the database—runs on an on-premises via Ollama, ensuring all document embeddings and inference remain within MSU-IIT's infrastructure to uphold institutional data-privacy requirements (Philippine Data Privacy Act compliance). Ollama's lightweight local runtime provides cloud-like elasticity—seamlessly scaling to accommodate peak query loads, while exposing inference endpoints through Django's REST framework. This integration delivers low-latency, auditable API calls for document retrieval, fits naturally into CatSight AI's unified agent architecture, and aligns with MSU-IIT's governance policies around data sovereignty server with Docker Compose, so all documents, extracted text, embeddings, and conversations stay on institutional hardware. Surya OCR 2, the smallest model, also runs on this server, on its integrated GPU through llama.cpp. The language, embedding, and reranking models are reached through OpenRouter's OpenAI-compatible API (OpenRouter, 2026), because the proponents have no GPU server to host them; each request carries only the page region, passages, or question being processed, and the documents themselves are stored only on the server.

Because every model is open-weight, the institution can move them onto its own GPU server later by pointing the same settings at a self-hosted, OpenAI-compatible endpoint (for example, vLLM or llama.cpp). Table 4.4 estimates the GPU memory each model needs. A single 80 GB data-center GPU (NVIDIA A100 or H100) can serve all four models at once if Qwen3-VL is served in 8-bit precision; two 24 GB GPUs (for example, NVIDIA L4 or RTX 4090) can serve them with Qwen3-VL in 4-bit precision.

Table 4.4. Estimated GPU Requirements for Self-Hosting the Models

Model	Weights (16-bit)	Comfortable GPU	Smaller option
Surya OCR 2	≈1.4 GB	Any 8 GB GPU (e.g., RTX 3060, RTX 4060)	Runs on a CPU, slowly
Qwen3-VL-30B-A3B-Instruct	≈62 GB	1× A100 or H100, 80 GB	8-bit (≈31 GB): 1× L40S, 48 GB; 4-bit (≈17 GB): 1× RTX 4090, 24 GB
BGE-M3	≈1.1 GB	Any 4 GB GPU (e.g., T4)	Runs well on a CPU
Qwen3-Reranker-8B	≈16 GB	1× 24 GB GPU (L4, A10G, RTX 4090)	Qwen3-Reranker-4B or -0.6B

Note. Weights ≈ parameters × bytes per parameter (2 bytes at 16-bit precision). “Comfortable” leaves about 20–30% of GPU memory for the attention cache of concurrent requests.

4.6 Development of the System

This section outlines the technical implementation of the system. Under this section are the dependency management, system algorithm, system functions, environment, the development environment, the architecture, the algorithms, the workflow of a request, the system’s functions, and the environment and resources that underpin the system its operation.

Dependency Management and Setting up Development Environment

4.6.1 Dependency Management and Development Environment

CatSight AI leverages **Docker**—CatSight AI uses Docker to package the application and all its dependencies into a containerized environmentcontainers. Docker is an open platform for developing, shipping, and running applications in lightweight, self-contained units called containers. By defining a **Dockerfile** that (Docker, Inc., 2024). A Dockerfile lists the base image, required system packages, Python runtime, and Python libraries(in a ‘requirements.txt’), every developer and deployment environment uses the exact; Docker Compose then starts the services of Figure 4.3 together with one command. Every developer and the server therefore use the same setup, eliminating “it works on my machine” issues. Containers isolate CatSight AI’s runtime from host-level variations, streamline onboarding (one ‘docker build’ + ‘docker run’ command), and support consistent CI/CD pipelines across development, testing, and production

stages (Doeker, 2024) The backend has an automated test suite that replaces every model with a deterministic stand-in, so it runs without network access or API costs, and the frontend is type-checked with TypeScript before each build.

4.6.2 System Architecture

Figure 4.3 shows the components of CatSight AI and how they interact. The numbered interactions are:

- (1) The user's browser loads the React single-page application and calls the API over HTTPS. The server publishes only one port, behind a secure tunnel.
- (2) nginx serves the application's static files and forwards API requests to the Django REST API. Chat answers stream back as Server-Sent Events.
- (3) When a document is uploaded, the API stores the file, records the document, and places a processing task on the Redis queue.
- (4) A Celery worker takes the task and runs the document pipeline (Algorithm 1).
- (5) The worker and the API read and write PostgreSQL, which stores the documents, their extracted text, the passages with their embeddings (pgvector) and full-text vectors, the users, and the chat histories with the agent's saved state.
- (6) The chat agent inside the API calls the language model, the embedding model, and the reranker.
- (7) The worker sends hard page regions to the vision model (Marker's LLM mode), the extracted text to the language model for cataloguing, and the passages to the embedding model.
- (8) Marker sends every page region to Surya OCR 2, which llama.cpp serves on the server's own GPU, outside the containers.

~~System Algorithm~~

4.6.3 System Algorithms

The core logic of the system is captured in ~~three complementary algorithms: the Document Process Algorithm~~ six algorithms. Every uploaded document moves through the stages of Fig-

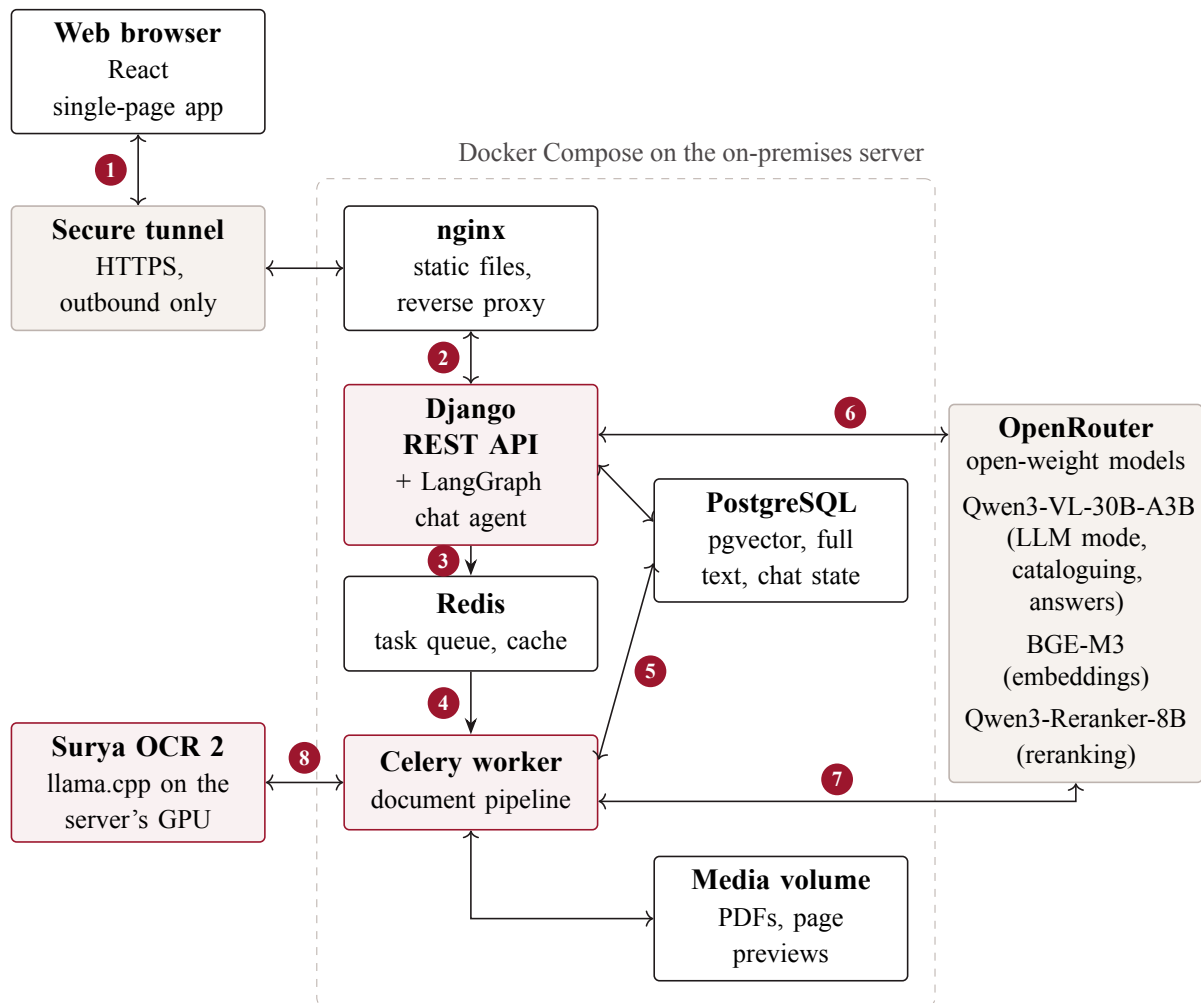


Figure 4.3. *System Architecture of CatSight AI*

ure 4.4 under Algorithm 1, which validates the file, rejects exact duplicates by their SHA-256 hash, and runs three stages: *extracting* the text of each page with Marker (Algorithm 2), *summarizing* the document into a catalogue entry (Algorithm 3, drawn in Figure 4.5), and *indexing* its passages (Algorithm 4). A failure stops the document at the failed stage with a readable message, and reprocessing resumes there instead of starting over. Questions are answered by the search-retrieval algorithm (Algorithm 5, drawn in Figure 4.6), which serves both the search page and the chat agent, and by the interactive-chat algorithm (Algorithm 6, whose states appear in Figure 4.7).

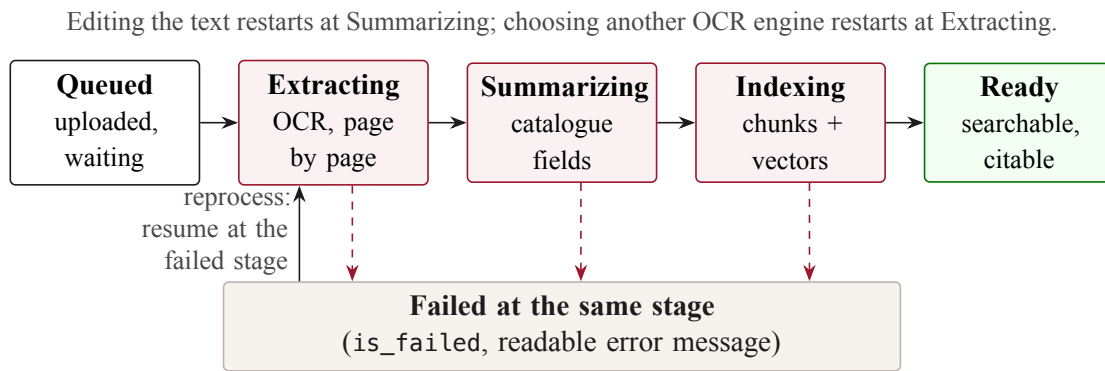


Figure 4.4. *Stages of the Document Pipeline*

Algorithm 1: *Document Processing*

Input: an uploaded PDF file f ; the uploading user u
Output: a searchable document d , or the reason f was not accepted

- 1 $info \leftarrow \text{Inspect}(f)$; $\triangleright$ PDF signature, file size and page limits
- 2 **if** $info$ is invalid **then return** rejected;
- 3 $h \leftarrow \text{SHA-256}(f)$;
- 4 **if** a document visible to u already has hash h **then return** duplicate;
- 5 $\text{ConsumeQuota}(u, info.pages)$; $\triangleright$ public-demo limits, under a database lock
- 6 $d \leftarrow$ new document with status Queued; store f and a first-page preview;
- 7 enqueue $\text{Process}(d)$ on the task queue;
- 8 **function** $\text{Process}(d)$
- 9 **foreach** stage $s \in \langle \text{Extracting}, \text{Summarizing}, \text{Indexing} \rangle$, starting at $d.status$ **do**
- 10 record the transition of d to s ;
- 11 run stage s : Extracting: $d.text \leftarrow \text{ExtractPages}(f)$; $\triangleright$ Algorithm 2
- 12 Summarizing: $d.catalogue \leftarrow \text{GenerateSummary}(d.text)$; $\triangleright$ Algorithm 3
- 13 Indexing: $d.chunks \leftarrow \text{IndexDocument}(d)$; $\triangleright$ Algorithm 4
- 14 **if** the stage raised an error e **then**
- 15 mark d failed at s with a readable message for e ;
- 16 **return**; $\triangleright$ a later reprocess resumes at s
- 17 record the transition of d to Ready;

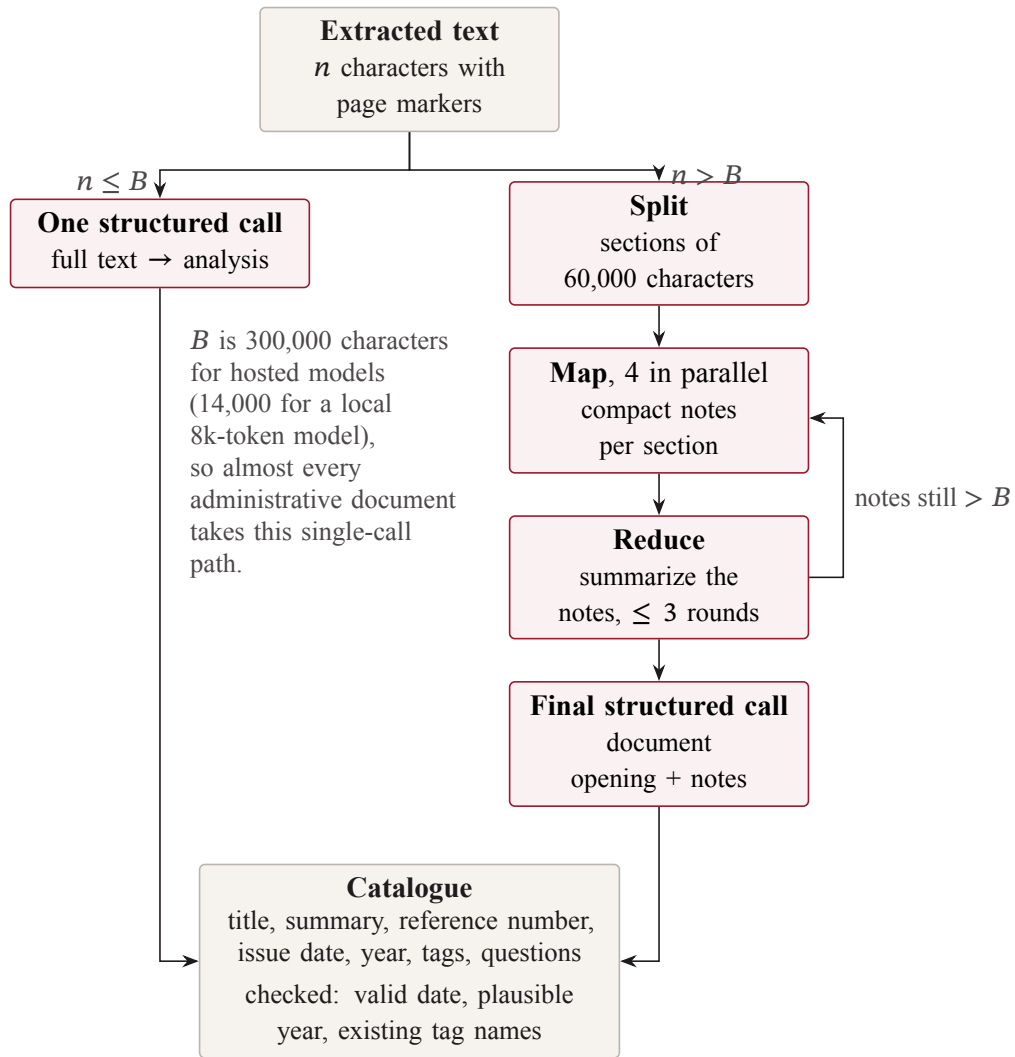


Figure 4.5. *Summarization Flow: One Structured Call, or Map-Reduce for Long Text*

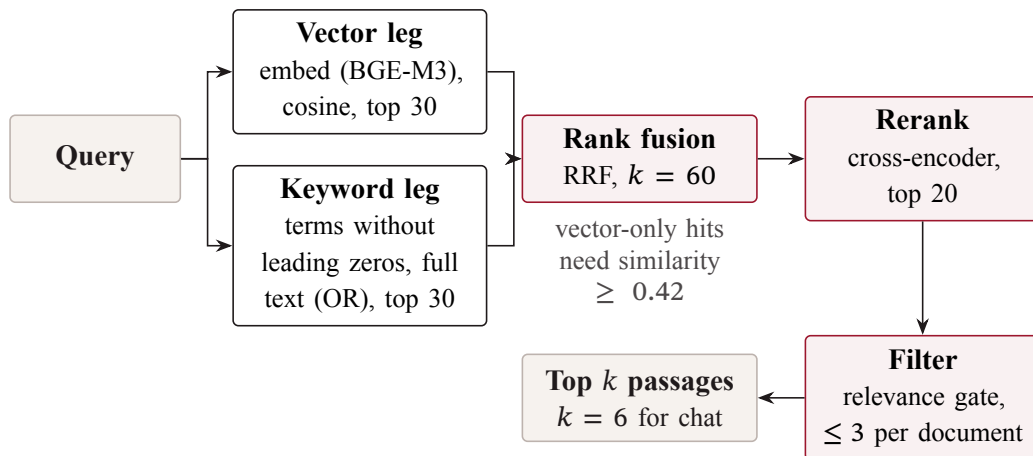
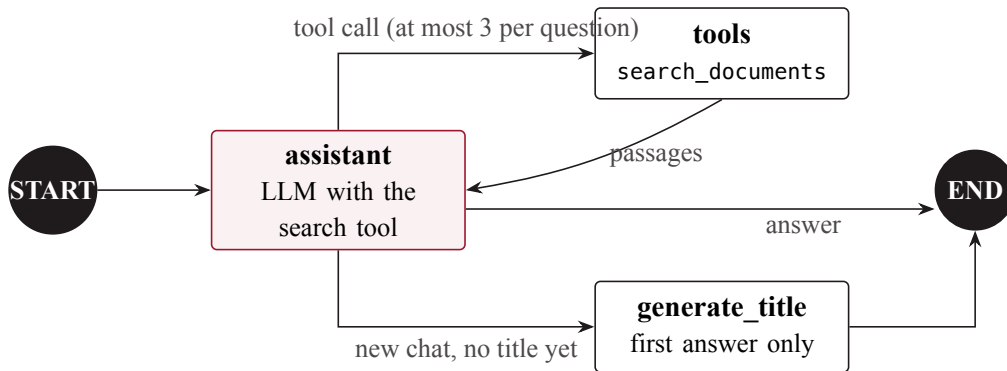


Figure 4.6. *Hybrid Search-Retrieval Flow*

Algorithm 6: Interactive Chat

Input: question x in chat c ; optional document scope D
Output: a streamed answer with numbered citations, and the numbered sources

- 1 $H \leftarrow$ the last 12 questions and answers of c , without earlier search results;
- 2 $sources \leftarrow \langle \rangle$; $searches \leftarrow 0$;
- 3 **repeat**
- 4 $y \leftarrow$ LLM(instructions, H , x , results so far) with tool `search_documents` if $searches < 3$, required when $searches = 0$ unless x is a greeting;
- 5 **if** y calls `search_documents( $q'$ )` **then**
- 6 **if** D is one document of fewer than 60,000 characters **then**
- 7 $R \leftarrow$ all passages of D in reading order ; $\triangleright$ read whole
- 8 **else**
- 9 $R \leftarrow$ HybridSearch(q' , scope D , $k = 6$) ; $\triangleright$ Algorithm 5
- 10 number each new document of R after those in $sources$; keep earlier numbers;
- 11 send the events `search( $q'$ )` and $sources$ to the browser;
- 12 $searches \leftarrow searches + 1$;
- 13 **until** the model answers without calling the tool;
- 14 stream the tokens of y to the browser as they arrive;
- 15 **if** c has no title **then** $c.title \leftarrow$ FastLLM(x);
- 16 save the conversation state of c ;



State: messages, chat title, optional document scope; saved after every step in PostgreSQL (one thread per chat)

Figure 4.7. *The Chat Agent as a State Graph*

4.6.4 System Workflow

Figure 4.8 follows one chat question through the components of Figure 4.3. The browser posts the question; the API runs the agent's graph for the chat; the language model asks for a search; the search embeds the query, runs the vector and keyword searches in PostgreSQL, and reranks the candidates; the numbered sources go back to the model and, at the same time, to the browser, which can show them before the answer is written. The model then writes the answer from those passages, and its tokens stream to the browser as they are produced. Finally, the agent's state is saved, so the conversation can continue later, and a short title is generated for a new chat.

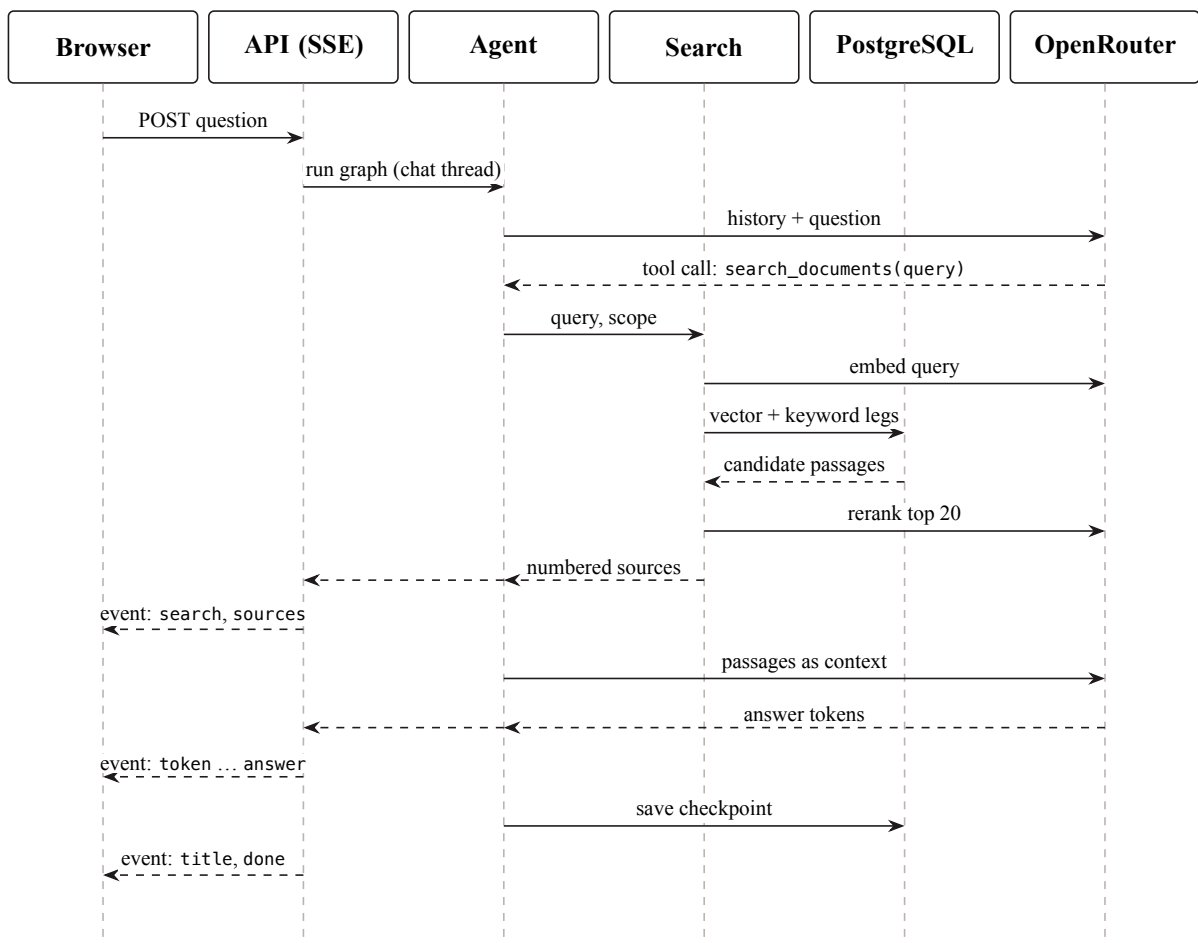


Figure 4.8. *Sequence of One Chat Question*

4.6.5 System Functions

This section presents the functions of CatSight AI through screenshots of the deployed system. Visitors open the public landing page (Figure 4.9) and enter with one click as a temporary guest, or sign in with an account (Figure 4.10).

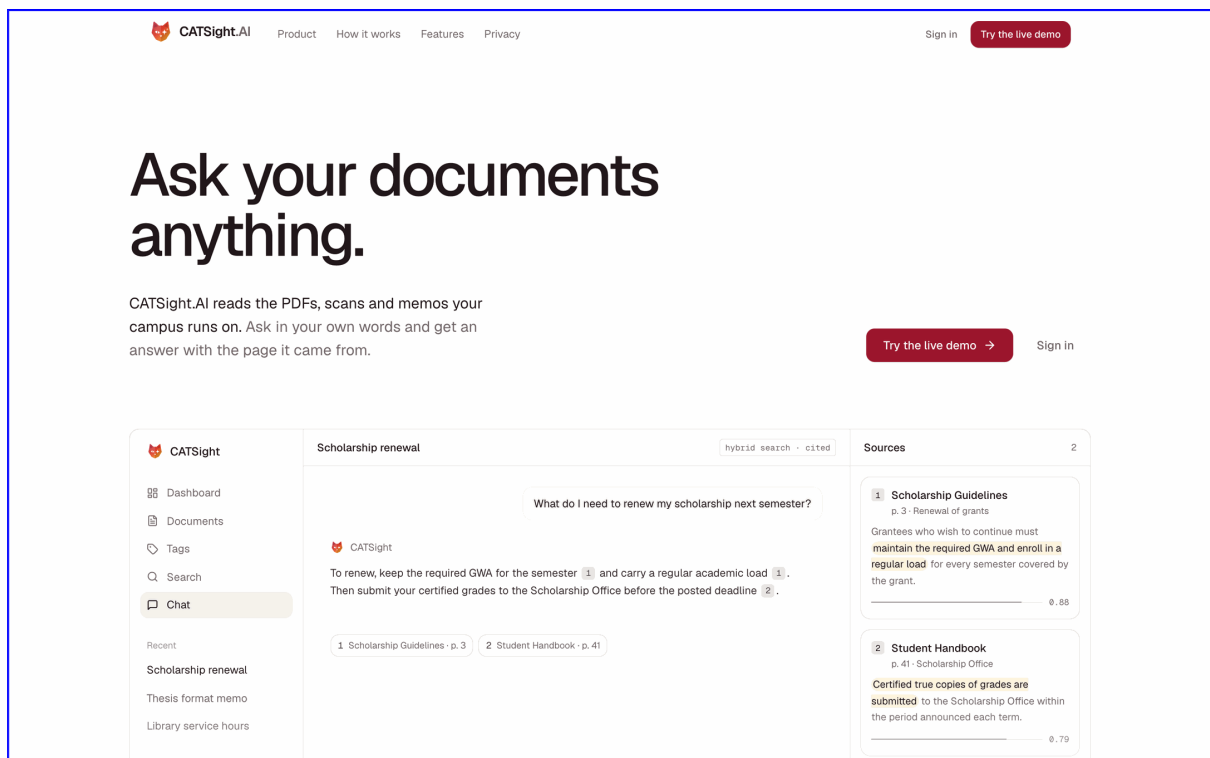


Figure 4.9. *Landing Page*

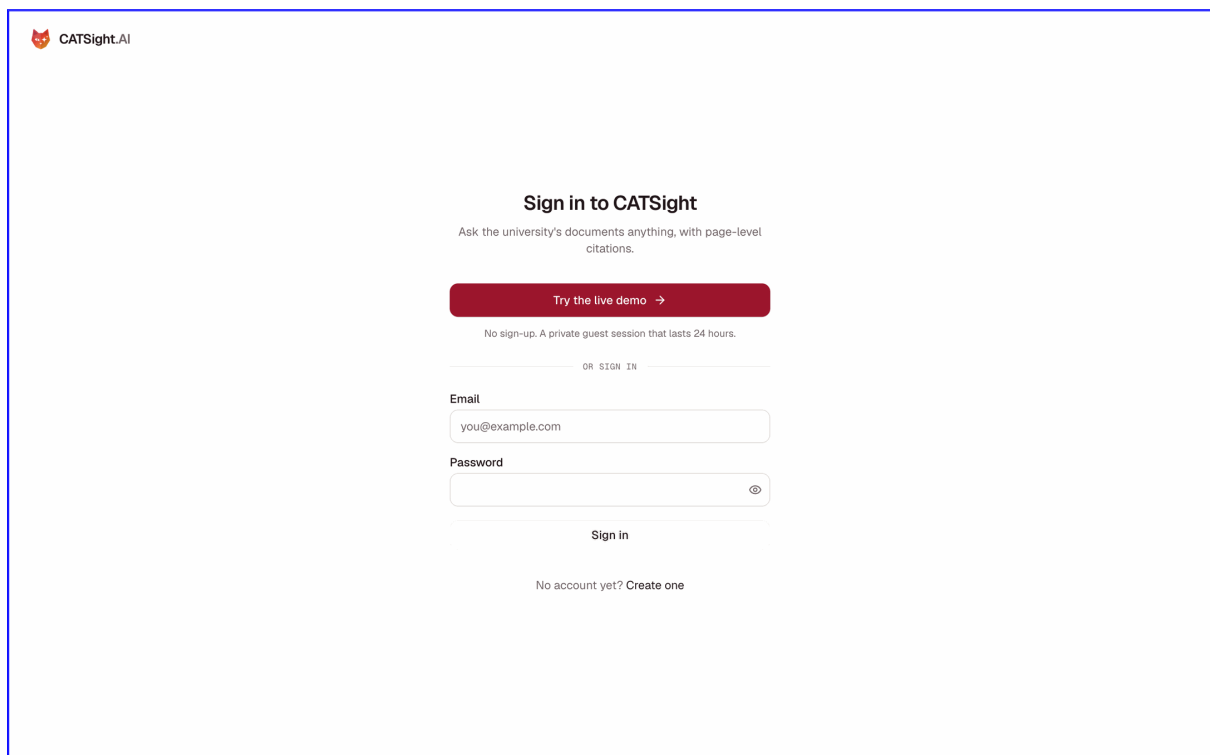


Figure 4.10. *Sign-In Page with One-Click Guest Access*

Dashboard. The dashboard (Figure 4.11) opens with a question box and suggested questions written by the cataloguing model, followed by the size of the library, the ingestion pipeline with the number of documents at each stage, and the distribution of documents across tags.

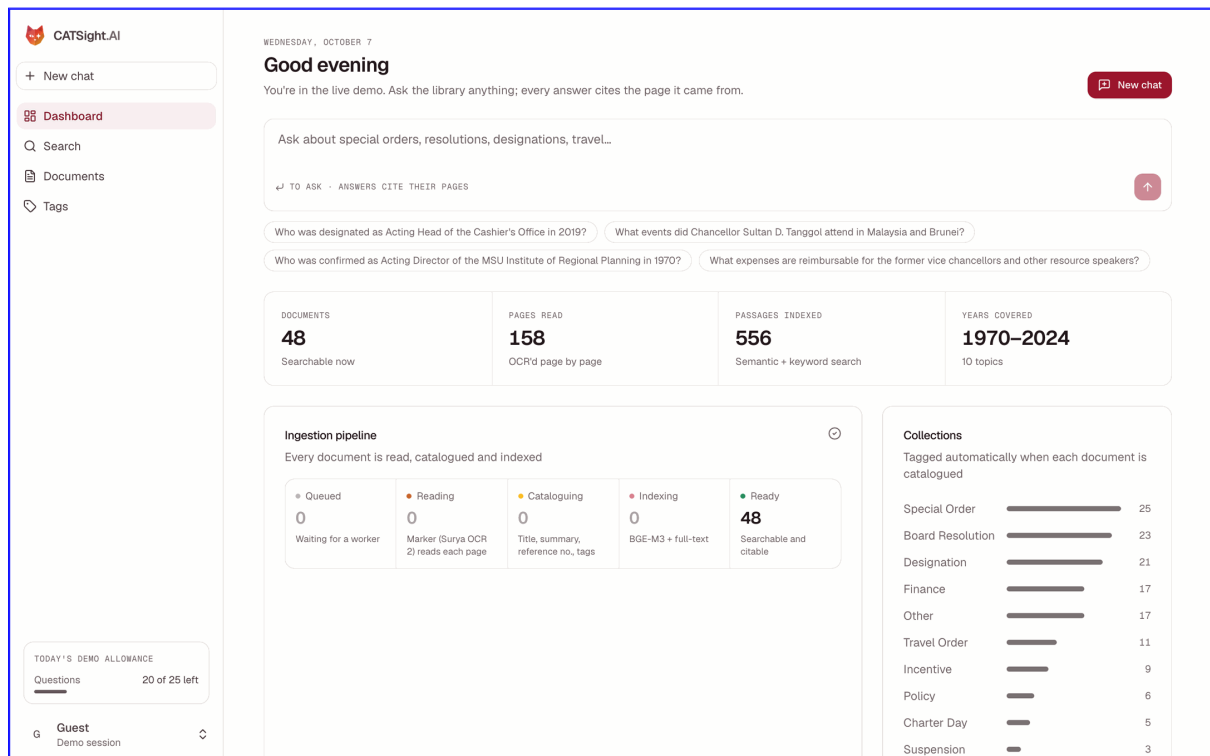


Figure 4.11. *Dashboard*

Uploading documents. Administrators upload PDFs by dragging them onto the upload dialog (Figure 4.12). Each file is checked, deduplicated, and queued; its progress then appears on the dashboard and on the document's page. In the public demonstration, visitors cannot upload.

Processing. While documents are processed, the dashboard's pipeline card shows how many are at each stage and which ones are moving, with a progress bar for each (Figure 4.13); each document's page shows the same stages for that document (Figure 4.14). Both update by themselves.

Document library. The library (Figure 4.15) lists every document with its title, reference number, year, tags, page count, and status, and can be searched and filtered by tag, year, and status.

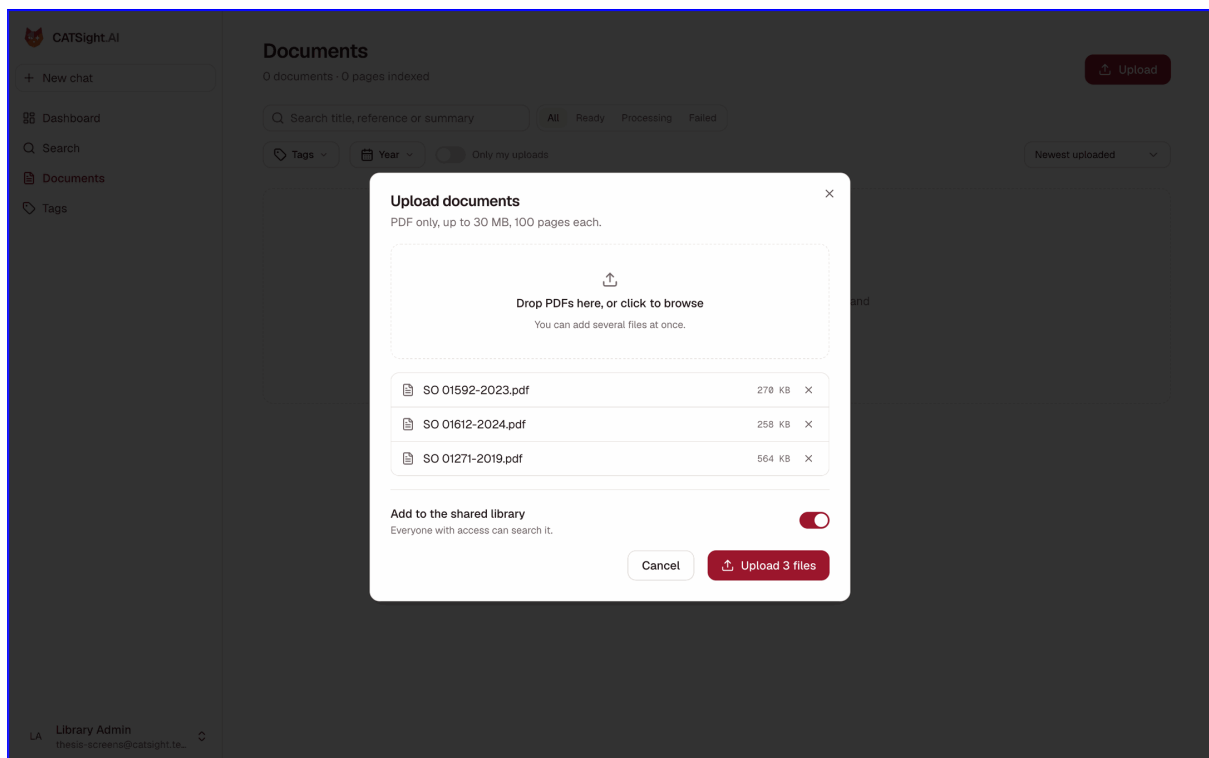


Figure 4.12. *Upload Dialog with Three Special Orders Selected*

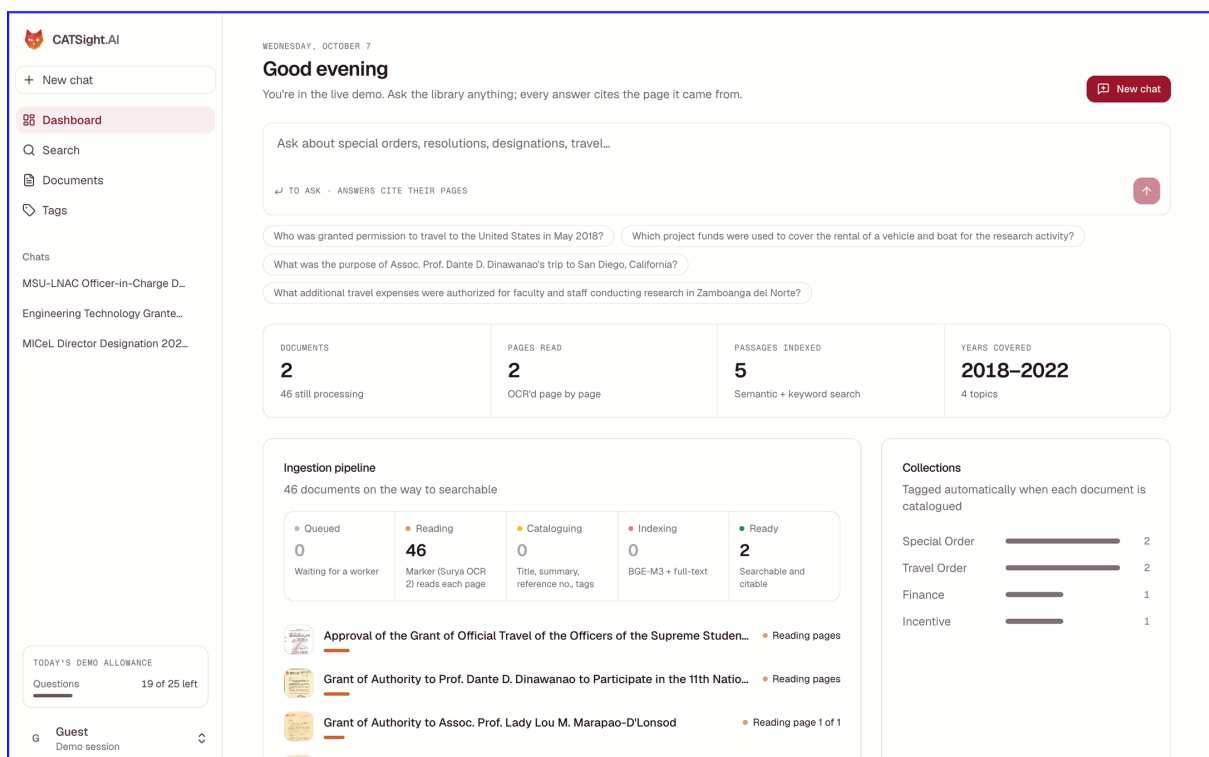


Figure 4.13. *The Ingestion Pipeline on the Dashboard While the Library Is Processed*

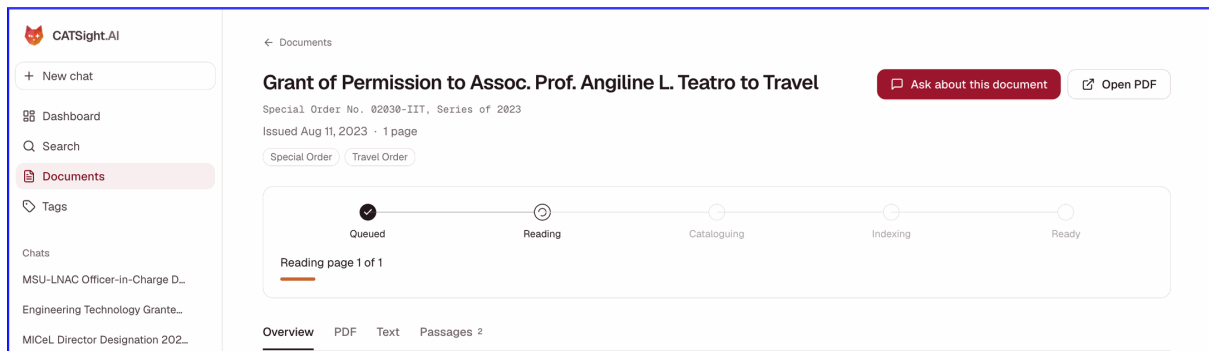


Figure 4.14. *A Document at the Reading Stage*

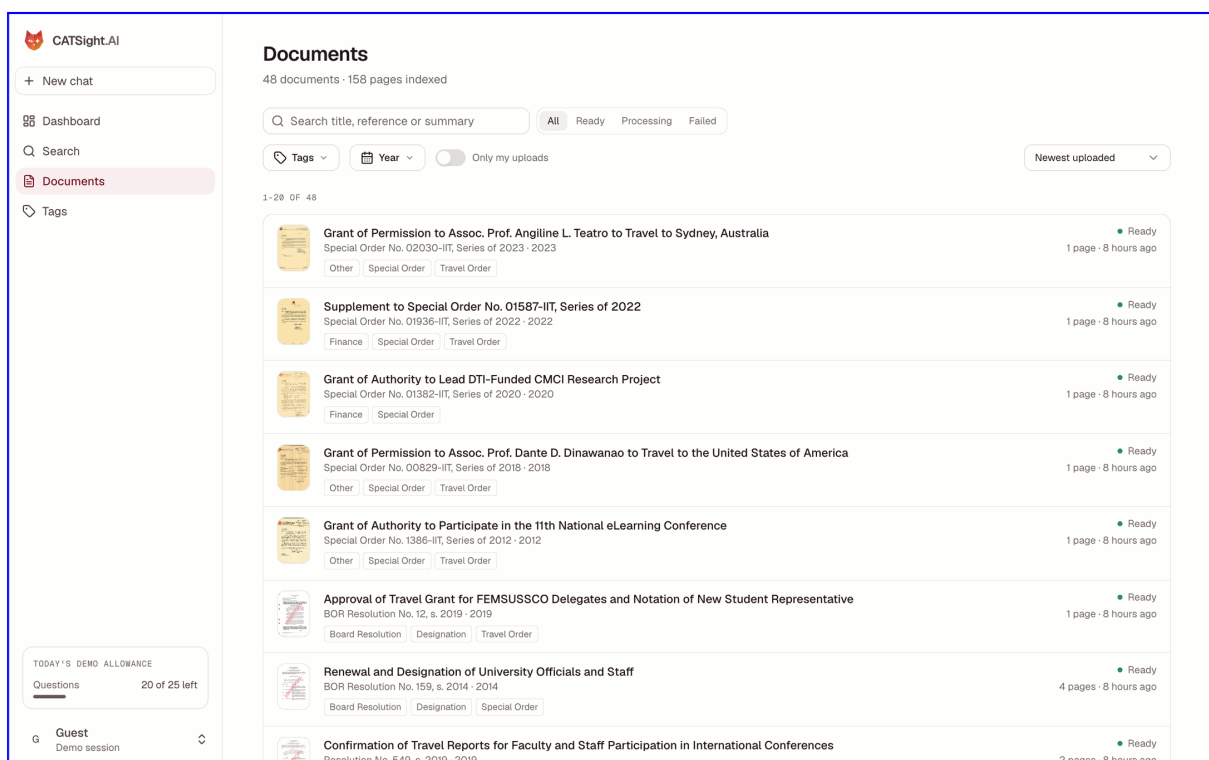


Figure 4.15. *Document Library*

Document details. Each document has four tabs. *Overview* (Figure 4.16) shows the catalogue written by the model—summary, reference number, issue date, tags, and two questions the document answers—along with processing details and the history of its pipeline stages. *PDF* (Figure 4.17) displays the original file. *Text* (Figure 4.18) shows the Markdown extracted by Marker, which an administrator can correct. *Passages* (Figure 4.19) lists the indexed passages with their page numbers.

The screenshot shows the CATSight.AI interface. On the left is a sidebar with navigation links: 'New chat', 'Dashboard', 'Search', 'Documents' (selected), and 'Tags'. The main content area is titled 'Documents' and displays a document titled 'Adjustment of Grants-in-Aid and Other Forms of Institute Scholarships and Financial Incentives for MSU-Iligan Institute of Technology'. Below the title, it shows 'BOR Resolution No. 169, s. 2003' and 'Issued Oct 17, 2003 · 9 pages'. There are three tags: 'Board Resolution', 'Finance', and 'Incentive'. At the top right, there are buttons for 'Ask about this document' and 'Open PDF'. Below the title, there are tabs for 'Overview', 'PDF', 'Text', and 'Passages 32'. The 'Overview' tab is active, showing a 'SUMMARY' section with a paragraph and a bulleted list of details: Who, What, When, Where, Amounts, Effectivity, and Conditions. To the right of the summary is a 'DETAILS' table with rows for Pages (9 pages), Passages (32), File size (214 KB), Extractor (Marker), Summarized by (Qwen3 VL 30B A3B L.), Uploaded by (Michael James Carnaje), Uploaded (Oct 7, 2026), and Processed (Oct 7, 2026 21:06). At the bottom, there is a 'QUESTIONS THIS DOCUMENT ANSWERS' section with two questions and their corresponding answers. On the far left, there is a 'TODAY'S DEMO ALLOWANCE' section showing 'Questions' and '20 of 25 left'. At the bottom left, it says 'Guest Demo session'.

Figure 4.16. *Document Overview: the Catalogue Generated from the Document*

Chat. The chat page (Figure 4.20) offers suggested questions. An answer (Figure 4.21) shows which searches the agent ran, leads with the direct answer, and marks each claim with a numbered citation; the cited documents are listed beneath it. Selecting a citation opens the source passage with its page (Figure 4.22).

Tags and settings. Tags (Figure 4.23) organize the library; each has a description that the cataloguing model uses to choose tags. Settings (Figure 4.24) shows the account, the light or dark theme, the user's daily allowance, and the models the deployment uses.

Phones. The interface adapts to phone screens (Figure 4.25).



Figure 4.17. *The Original PDF*



Figure 4.18. *Text Extracted by Marker*

+ New chat

Dashboard

Search

Documents

Tags

TODAY'S DEMO ALLOWANCE

Questions 20 of 25 left

Guest Demo session

← Documents

Adjustment of Grants-in-Aid and Other Forms of Institute Scholarships and Financial Incentives for MSU-Iligan Institute of Technology

Ask about this document

Open PDF

BOR Resolution No. 169, s. 2003

Issued Oct 17, 2003 · 9 pages

Board Resolution Finance Incentive

Overview PDF Text Passages 32

Filter passages

32 PASSAGES

#0 p. 1 Republic of the Philippines MINDANAO STATE UNIVERSITY Marawi City > OFFICE OF THE SECRETARY OF THE UNIVERSITY AND OF THE BOARD OF REGENTS

Republic of the Philippines MINDANAO STATE UNIVERSITY Marawi City

OFFICE OF THE SECRETARY OF THE UNIVERSITY AND OF THE BOARD OF REGENTS

EXCERPTS FROM THE MINUTES OF THE 188TH MEETING OF THE BOARD OF REGENTS HELD AT SEAMEO-INNOTECH, COMMON WEALTH AVENUE, DILIMAN, QUEZON CITY ON OCTOBER 17, 2003

X-X-X

X-X-X

X-X-X

#1 p. 1 "RESOLUTION NO. 169, S. 1. 03"

"RESOLUTION NO. 169, S. 1. 03"

RESOLVED, That upon the recommendation of the President of the University and the Executive Commission of the Board of Regents, the proposed adjustment of Grants-in-Aids and Other Forms of Institute Scholarships and Financial Incentives as contained in the Brief of the Vice Chancellor for Academic Affairs to the Chancellor of MSU-Iligan Institute of Technology, which is attached here and marked as Annexes "K" to "K-8" and made an integrated part of this resolution, is hereby approved.

Figure 4.19. *Indexed Passages with Their Page Numbers*

+ New chat

Dashboard

Search

Documents

Tags

TODAY'S DEMO ALLOWANCE

Questions 19 of 25 left

Guest Demo session

What would you like to know?

CATSight searches the special orders, resolutions and memos in the library, then answers with numbered citations to the exact pages.

Who was designated Chairperson of the Department of Civil Engineering at MSU-IIT in 2011?

Who was designated Acting Executive Director of the ICTC at MSU-IIT in 2003?

What sub-committees were created for the 2002 Charter Day Celebration?

Who was granted permission to travel to the United States in May 2018?

Ask about special orders, resolutions, designations, travel...

All documents

↑

Answers come only from the documents and cite their pages; check important details in the source. · 19 questions left today

Figure 4.20. *Chat with Suggested Questions*

71

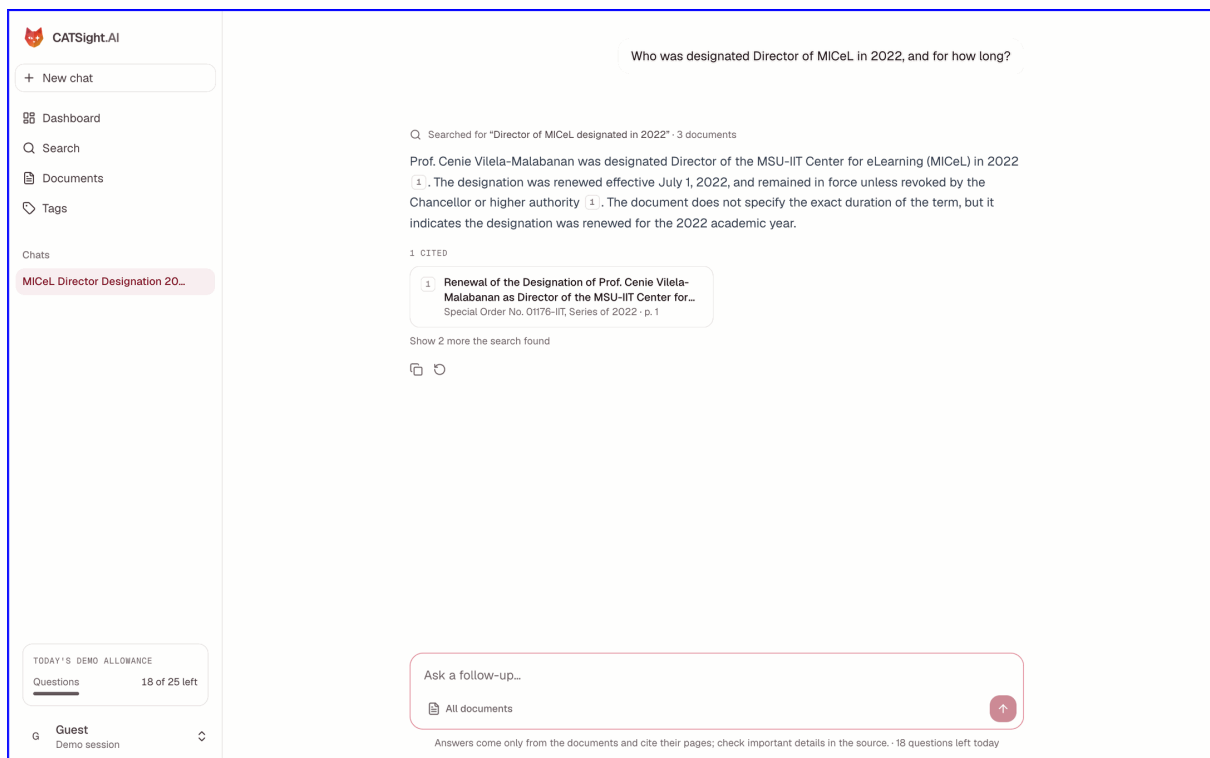


Figure 4.21. *A Cited Answer*

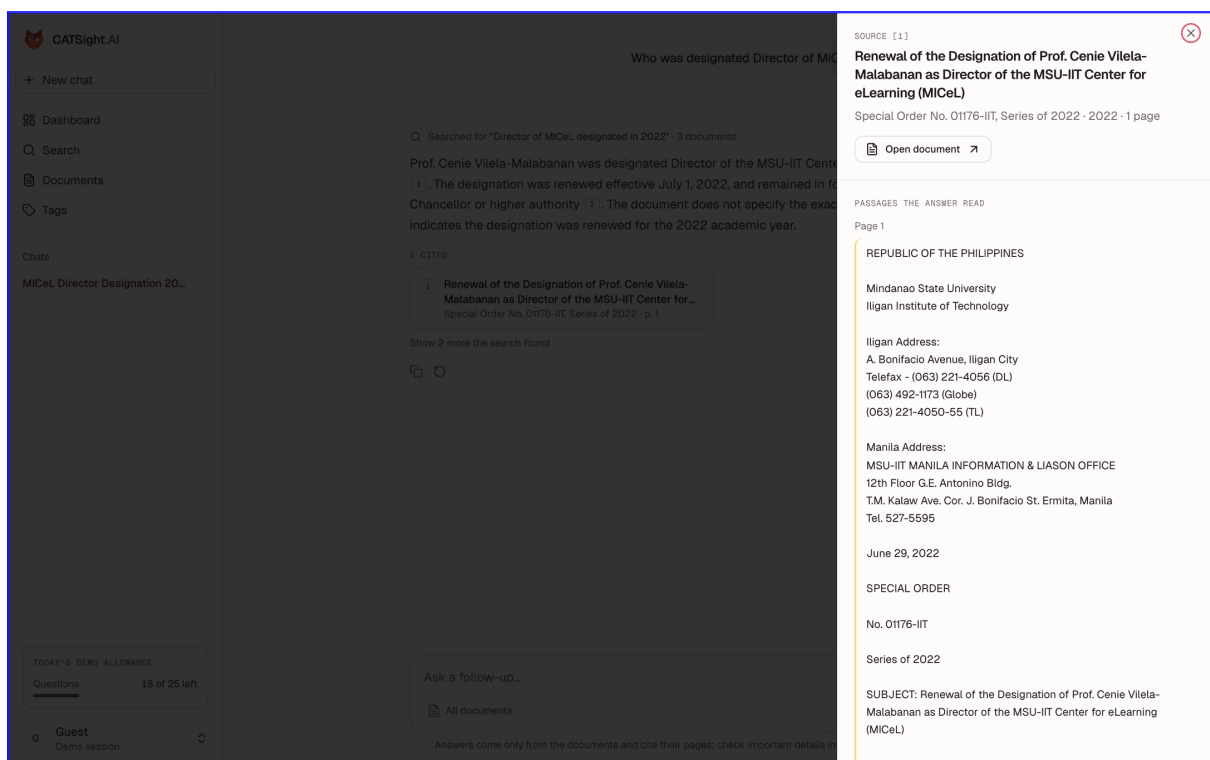


Figure 4.22. *A Citation Opened to Its Source Passage*

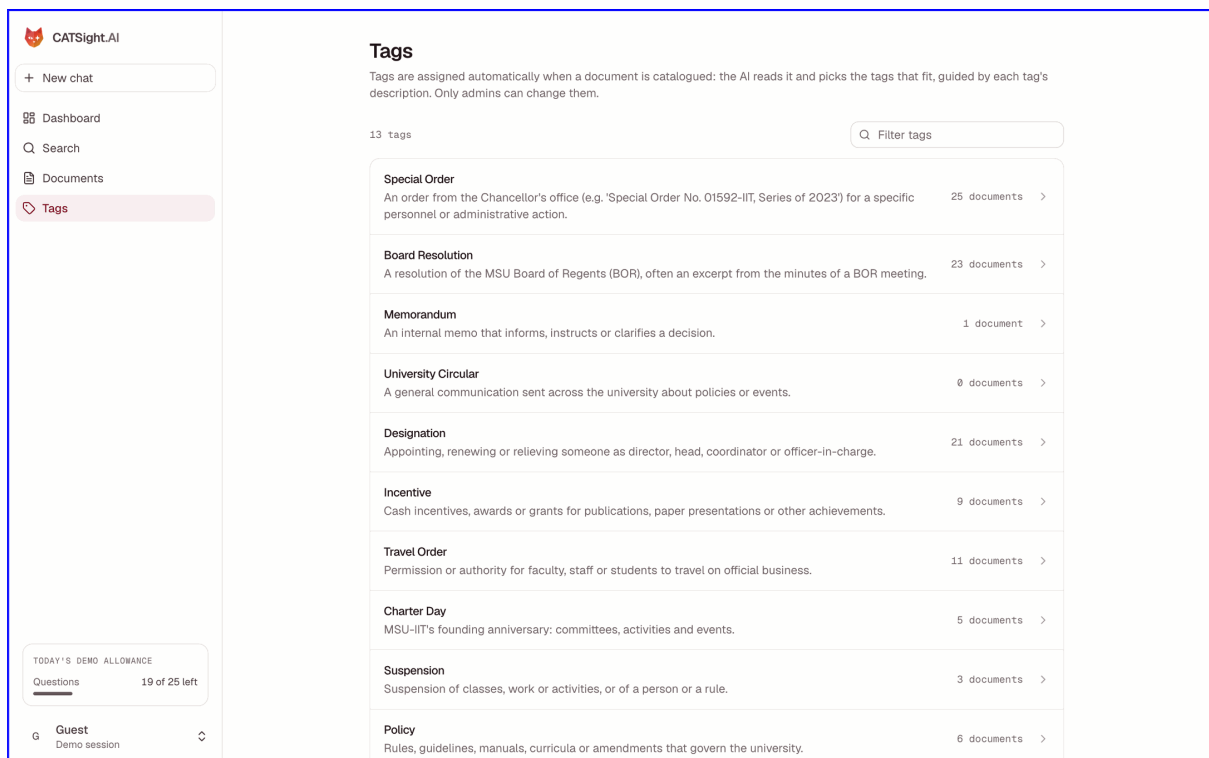


Figure 4.23. *Tags*

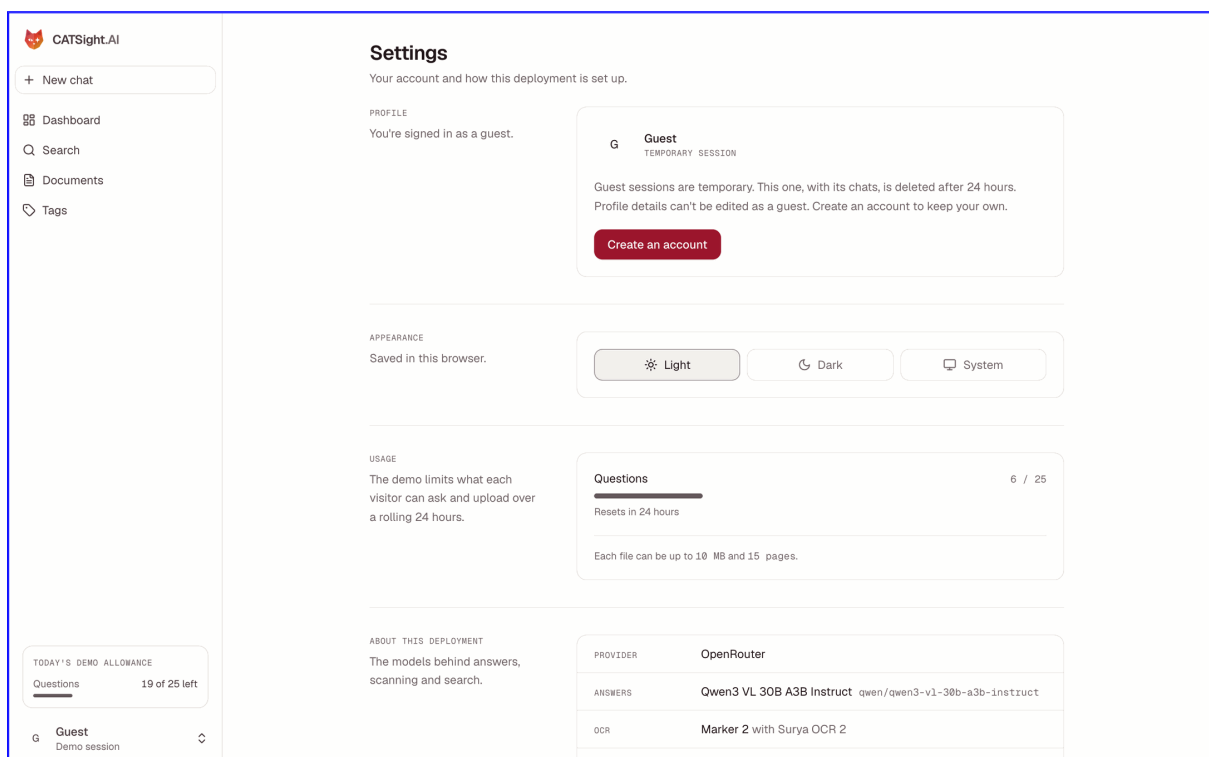


Figure 4.24. *Settings*

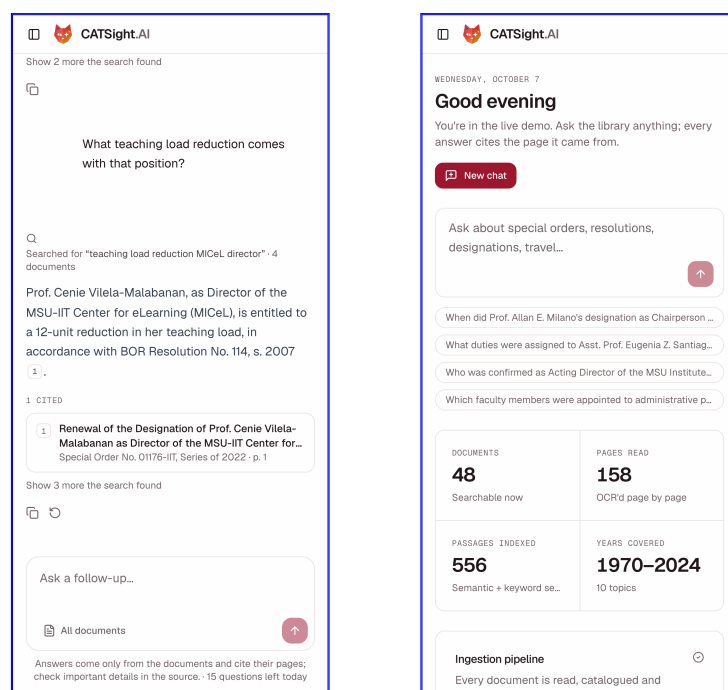


Figure 4.25. *Chat and Dashboard on a Phone*

4.6.6 Environment and Resources

Table 4.5 lists the software CatSight AI is built with. The deployed system runs on a small on-premises server with an integrated GPU, which serves the OCR model; its containers, which cannot use the GPU, are allocated 10 CPU cores and 8 GB of memory. Development and the converter comparison of Section 6.1 used a laptop with an Apple M4 Pro processor and 24 GB of memory.

Table 4.5. *Software Used to Build CatSight AI*

Layer	Software
Interface	React 18, TypeScript 5.6, Vite 6, Tailwind CSS 3.4, shadcn/ui, TanStack Query 5, react-pdf 9 (PDF.js)
API	Python 3.11, Django 5.1, Django REST Framework 3.15, gunicorn 23, Server-Sent Events
Document pipeline	Celery 5.5, Redis 7, pypdfium2 4.30, Marker 2.0 with Surya OCR 0.22, llama.cpp (build b11429), PyTorch (CPU)
Retrieval and agent	LangChain 1.x, LangGraph 1.x with a PostgreSQL checkpointer
Database	PostgreSQL 17 with pgvector and full-text search
Models	Surya OCR 2, Qwen3-VL-30B-A3B-Instruct, BGE-M3, Qwen3-Reranker-8B (Table 4.3) through OpenRouter; Ollama for the local configuration
Converters compared	Docling 2.134, MarkItDown 0.1.8
Deployment	Docker Compose, nginx, an outbound-only HTTPS tunnel

4.7 Evaluation Procedure

The system is evaluated with the measures defined in Section 3.3.

1. **Text extraction.** The four converters supported by CatSight AI and two vision-model variants are run on a sample of seven documents (26 pages) chosen to cover pure scans, partial text layers, digital files, a watermark, tables, and a typewritten page from 1970. For each document, 8 to 11 facts (reference numbers, names, dates, amounts) were read from the page images, and a converter is credited with a fact when it appears in its output, ignoring case, spacing, punctuation, and leading zeros. Time per page is also recorded.
2. **Retrieval accuracy.** A set of test questions, each with the documents that answer it, is run through CatSight AI's search and through the keyword search of IIT Docs; P@5, R@5, and MRR (Equations 3.8 and 3.9) are compared. **[TODO: Number of test questions and how the relevant documents were judged.]**
3. **Summary quality.** For a sample of documents, a person writes a reference summary without seeing the generated one; ROUGE-1, ~~which the user uploads files which will then be converted into Markdown, then from Markdown~~ ROUGE-2, ~~it will be broken down using RecursiveTextSplitter from Langchain for context-aware retrieval~~ ROUGE-L, BLEU, and BERTScore (Equations 3.10–3.13) compare each generated summary with its reference. **[TODO: Number of documents and who wrote the references.]**
4. **Efficiency.** Search time is reported by the server for each search; time to first token and total response time are measured for chat answers; ingestion time is measured per page.
5. **Usability.** Participants complete tasks with the existing system and with CatSight AI and then answer the SUS questionnaire (Equation 3.14) for each.

Chapter 5 Design and Implementation Issues

This chapter discusses the design and implementation of the major data structures and algorithms used in CatSight AI. It describes the data model (Section 5.1) and the main data structures (Section 5.2), then the ~~retrieved-markdown-will-be-embedded~~ major issues encountered during development and the solutions and alternatives the proponents employed (Section 5.3, summarized in Table 5.1).

5.1 Data Model

Figure 5.1 shows the main tables of the PostgreSQL database and their relationships. A **Document** holds the catalogue fields written by the model (title, summary, reference number, issue date, year, suggested questions), the file and its SHA-256 hash, and the pipeline fields (stage, failure flag, error message, progress). Its extracted Markdown lives in a separate **Document-FullText** row, so that listing documents never loads full texts. A document has many **DocumentChunk** passages, each with its page, section, context header, text, a 1024-dimensional embedding, and a full-text vector with a GIN index. Every stage change is appended to **DocumentStatusHistory**, which drives the document’s timeline. **Tags** relate to documents many-to-many. A **User** has a role (guest, user, administrator, or super administrator), owns **Chats**, and accumulates **UsageEvents** for the demonstration’s daily limits. Each chat’s conversation state is stored by the agent’s checkpointer in its own tables, keyed by the chat’s thread identifier.

5.2 Major Data Structures

Page-marked Markdown. Every converter returns a list of pages, which are joined into one Markdown text with an invisible marker before each page (`<!-- page:3 -->`). The markers survive manual edits of the text, do not show when the Markdown is displayed, and let the chunker attach a page number to every passage whichever converter produced the text. Citations such as “p. 3” depend on them.

Passages with context. A passage alone is often ambiguous: “...is hereby designated as Director” does not say which order it belongs to. Each passage is therefore stored with a *context*

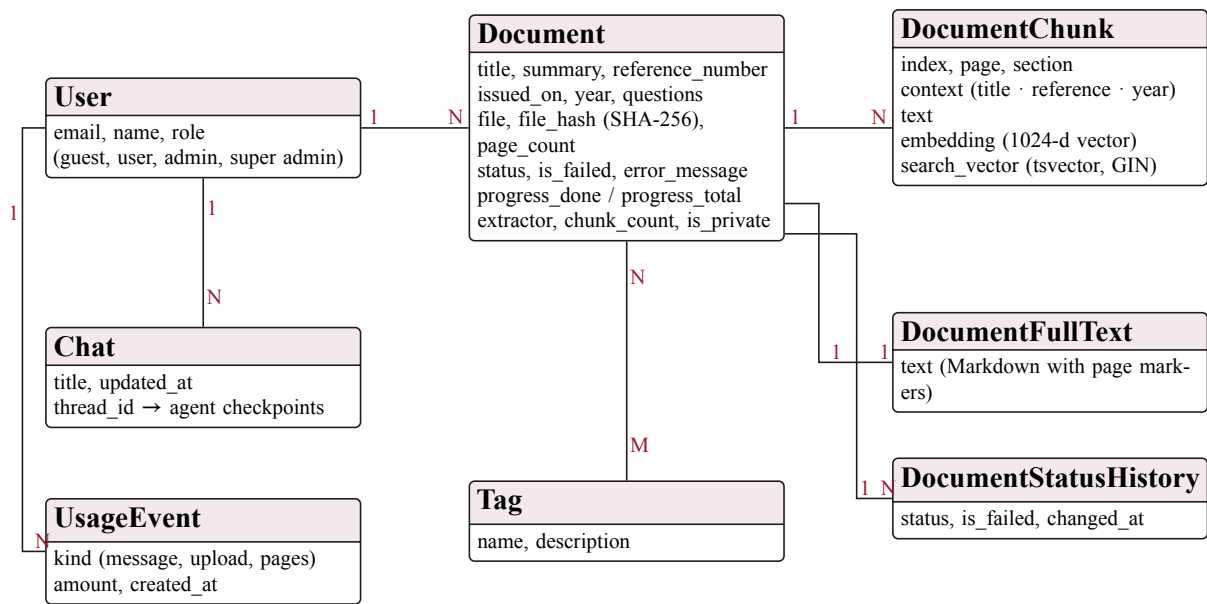


Figure 5.1. *Data Model of CatSight AI*

header—the document’s title, reference number, and year—which is prepended when the passage is embedded and indexed, with the header weighted above the body in the full-text vector. The chunker splits the text at headings first, so that a passage never spans two sections, and then into pieces of about 1,000 characters with a 150-character overlap.

Search hits. A search result carries the passage, its final score, the cosine similarity if the vector search found it, the keyword rank if the keyword search found it, and its rank in each list. The interface uses these to label each passage as a “related meaning” match, an “exact match”, or both.

Agent state and numbered sources. The agent’s state is the list of messages, the chat’s title, and an optional set of documents the user limited the chat to; it is saved after every step. Within one answer, each cited document receives a number the first time a search returns it and keeps that number across later searches, so the “[2]” in the text always refers to the second source shown in the interface.

5.3 Issues Encountered and Solutions

Table 5.1. *Major Design and Implementation Issues and Their Solutions*

Issue	Solution
Scanned pages, watermarks and tables	Marker 2 with Surya OCR 2, plus LLM mode for hard regions (5.3.1)
A CPU-only server for an OCR model	A separate llama.cpp service sized to avoid truncation (5.3.2)
Crashes from concurrent PDF rendering	One process-wide lock around PDFium (5.3.3)
Long tasks and lost work	Acknowledge late, long visibility timeout, resume at the failed stage (5.3.4)
Identifiers that embeddings blur	Hybrid search, leading-zero normalization, weighted context (5.3.5)
Irrelevant passages	A similarity floor and a reranker gate (5.3.6)
Approximate indexes dropping results	Exact vector search at this scale (5.3.7)
Documents longer than the context	Map-reduce cataloguing (5.3.8)
Unreliable structured output	Required fields, strict schemas, validation (5.3.9)
Hallucination and broken citations	Grounded instructions, stable numbering, low temperature (5.3.10)
Follow-ups answered without a search	A search required on every question but greetings (5.3.11)
A rates table the search missed	Short scoped documents read whole (5.3.12)
Answers without citation numbers	A reminder where the model reads the results (5.3.13)
Growing conversations	A window of recent turns without old search results (5.3.14)
Public demonstration costs	Guest accounts, atomic daily limits, a provider credit cap (5.3.15)

5.3.1 Reading Scanned Pages

Problem. Thirteen of the 50 files have no text layer, many of the rest carry poor OCR from the repositories, BOR resolutions are overlaid with a large diagonal “UNOFFICIAL” watermark, and several issuances contain tables of rates or amounts. A converter must read all of these faithfully, because a wrong digit in a reference number or a misspelled surname makes a document unfindable.

Solution and alternatives. Four converters and two vision-model variants were compared on a sample of seven documents (Section 6.1). Markdown only reads text layers and returned nothing for scanned pages. Docling’s standard pipeline missed much of the scanned

text. Sending page images straight to Qwen3-VL recovered most facts but showed two systematic errors: it omitted the date of issue printed above the reference number, and it shortened the surname “Vilela-Malabanan” to “Vilela-Malaban”—a silent error that propagated into the generated summary. Marker 2, whose Surya OCR 2 model is trained only to transcribe, copied that surname and the dates correctly. CatSight AI therefore uses Marker 2, ~~and the Interactive-Chat Algorithm, which enables users to engage in dynamic, conversational queries for more contextual and human-like interactions.~~ re-OCRs every page instead of trusting existing text layers, and enables Marker’s LLM mode so that Qwen3-VL corrects the tables, forms, and handwriting that line-level OCR handles worst. The other converters remain available as alternatives. Two details of Marker’s LLM mode also needed handling: with image extraction turned off, it wrote a description of every logo and seal into the text, where it would have been indexed as content, so images are left in and their links removed instead; and on signature regions the vision model sometimes fell into a loop, repeating blank lines. Marker then keeps its own OCR text for the region, but Marker sets no limit on the length of a reply, and because the provider keeps a long-running connection alive, its timeout never fired: one region held up a document for several minutes. Each reply is therefore capped at 4,096 tokens, which bounds a loop to about a minute.

5.3.2 Serving the OCR Model Without a GPU

Problem. Marker 2 does not run its OCR model inside Python: it starts a llama.cpp server and sends it page regions over an OpenAI-compatible API. Without that server program, Marker stops with an error. On the deployment server, the application’s containers cannot use the server’s GPU, and on a CPU the model needed about two minutes per page in testing (Section 6.6), which would make building the library take hours.

Solution. The OCR model runs outside the containers as a background service on the server’s GPU, started with the same options Marker would use: the model name Marker expects and a context of 12,288 tokens per slot, below which long pages are silently truncated. With two slots the small model left the GPU mostly idle and each page took about two minutes; six slots, about 0.55 GB of memory each, let it read six pages at once. It listens only on the server’s loopback interface, and a setting points Marker inside the containers to it. A containerized CPU

version of the same service remains available for servers without a usable GPU, and on a Linux GPU server the service can use a CUDA build of llama.cpp or vLLM without changing the application.

5.3.3 Crashes from Concurrent PDF Rendering

Problem. The worker processes up to four documents at once in threads. PDFium, the library that renders PDF pages, is not thread-safe: two threads rendering pages at the same moment, even of different files, crashed the whole worker process with a segmentation fault, and the documents it was processing stopped mid-stage.

Solution. Every call into PDFium goes through one process-wide lock. Rendering takes milliseconds, while the model requests that follow it run in parallel outside the lock, so the lock costs little. Marker and Docling call PDFium internally, so their conversions also run under this lock. Documents interrupted by the earlier crashes were requeued from their stage.

5.3.4 Long Tasks and Lost Work

Problem. Converting a long scanned resolution takes minutes. With the task queue's defaults, a task that ran longer than the broker's visibility timeout could be delivered to a second worker, and a worker restart could silently drop a task in progress.

Solution. Tasks are acknowledged only after they finish, the visibility timeout is raised to 30 minutes, and processing is *resumable*: each stage's result is saved before the next begins, a failure records the stage and a readable message, and reprocessing starts at that stage instead of from the beginning (Figure 4.4). Editing a document's text restarts it at summarizing; changing its converter restarts it at extracting.

5.3.5 Identifiers That Embeddings Blur

Problem. Most searches in an issuance repository name a reference number, a surname, or a year, and these are exactly what dense embeddings represent worst: "Special Order No. 01592" and "Special Order No. 01593" have almost identical embeddings. The repositories also write the same number inconsistently ("01592", "1592", "01592-IIT").

Solution. A keyword search over PostgreSQL full-text vectors runs alongside the vector search, and the two rankings are fused with Reciprocal Rank Fusion (Section 3.1.5). Leading

zeros are removed from numbers and hyphenated identifiers are split, both when indexing and when searching, so “SO 1592” finds “Special Order No. 01592-IIT”. The context header is weighted above the passage body, so a match on a document’s title or reference number counts for more.

5.3.6 Irrelevant Passages

Problem. Vector search always returns the nearest passages, even when nothing in the library is relevant, which invites the model to answer from unrelated text.

Solution. With BGE-M3, passages relevant to a question scored about 0.55 to 0.75 in cosine similarity on this corpus and unrelated passages about 0.30 to 0.40; a passage found only by vector search must therefore reach 0.42. After fusion, the reranker scores the top 20 candidates and passages below 0.05 are dropped—except exact keyword matches, which are kept even when the reranker is unsure, because an identifier match is strong evidence. At most three passages per document are returned, so one long document cannot crowd out the rest.

5.3.7 Approximate Indexes Dropping Results

Problem. Approximate nearest-neighbour indexes such as HNSW speed up vector search on large collections, but they search a fixed number of candidates first and apply filters (a user’s visible documents, a year, a tag) afterwards, so a filtered search can return fewer results than exist.

Alternative chosen. With a few hundred to a few thousand passages, an exact scan of all embeddings takes milliseconds, so CatSight AI uses no approximate index and never loses filtered results. An HNSW index can be added when the library grows to hundreds of thousands of passages.

5.3.8 Documents Longer than the Context Window

Problem. The longest document in the corpus has 36 pages, and with a local 8,192-token model even a six-page resolution exceeds what one call can read.

Solution. Cataloguing uses the map-reduce procedure of Section 3.2.4 (Algorithm 3): the text is split into sections, each section is condensed into notes in parallel, the notes are condensed again if needed, and the final call reads the document’s opening together with the

notes. With hosted models, whose context is large, almost every document is catalogued in one call.

5.3.9 Unreliable Structured Output

Problem. The catalogue must arrive as typed fields, but models occasionally omit optional fields, return the year as 0 or a page number, invent tag names, or write dates in different formats.

Solution. The JSON schema makes every field required, using null where a value may be absent, because models skip optional fields far more often than required ones; the request enforces the schema strictly. The application then validates the result: dates must parse, years must lie between 1900 and next year, and tags are kept only if they match an existing tag, ignoring case. Qwen3-VL also tended to condense a subject line into a generic title, dropping the names and positions people search for (“Grant of Permission to Travel to the United States for a Summit” instead of “Grant of Permission to Assoc. Prof. Dante D. Dinawanao to Travel to the United States of America”); the instruction now asks for the subject line word for word, including every name, position, place, and date.

5.3.10 Hallucination and Citation Integrity

Problem. Without safeguards, the model could answer from general knowledge, cite a number that matches no source, or renumber sources between searches so that citations point to the wrong document.

Solution. The instructions require the assistant to search before answering questions about the university, to answer only from the search results, to say plainly when they do not contain the answer, and to decline questions unrelated to the documents. Sources are numbered once per answer and keep their numbers across searches. Answers are generated at temperature 0.2. If the model returns an empty answer, it is asked once more before a polite failure message is shown. Section 3.2.2 gives the theoretical basis.

5.3.11 Follow-up Questions Answered Without a Search

Problem. The instructions tell the assistant to search before answering, but on a follow-up question—“What teaching load reduction comes with that position?”—Qwen3-VL answered

from the conversation instead. Because earlier search results are not shown to the model again (Section 5.3.14), it said the documents did not specify the reduction, and in a separate test it invented “2 units per semester”.

Solution. The first step of every question now requires the model to call the search tool; only short greetings, thanks, and questions about the assistant itself are exempt. The model still writes the query, so a follow-up is searched with the subject of the conversation (“teaching load reduction MICE director”), and the answer now gives the 12-unit reduction with its citation.

5.3.12 A Table the Search Did Not Find

Problem. In a chat limited to the 2003 grants-in-aid resolution, asked for the new maximum stipend of a full grantee of the School of Engineering Technology, the search returned three passages from the resolution’s rationale but neither the table of new rates nor the line “The highest amount of stipend, PhP 1,200.00”: both are worded unlike the question, and Marker had turned the “ANNEX K-2” stamps on those pages into headings, which became their section names. The model answered PhP 800, the old maximum quoted in the rationale.

Solution. A chat limited to one document of fewer than 60,000 characters (about 15,000 tokens) now reads the whole document in reading order instead of searching it, which nine pages of administrative text easily fit; longer documents are still searched, but without the usual limit of three passages per document. With the whole resolution in view, the answer is PhP 1,200.00, with its breakdown into meals and lodging.

5.3.13 Answers Without Citation Numbers

Problem. After the switch to Qwen3-VL, three of four answers in one session carried no citation numbers, although the instructions require them, so the interface had no passages to link.

Solution. The search results now end with a line naming the numbers to cite—the last thing the model reads before it answers—and the instructions close with the rule. In a check of eight questions, including a follow-up, a scoped question, and a question in Filipino, all eight answers carried citations.

5.3.14 Growing Conversations

Problem. Each search adds several passages to the conversation. Sending every earlier search

result with each new question would make long chats slow and expensive and would distract the model with old material.

Solution. Each ~~user~~ request includes the ~~input query along with session-specific metadata, enabling personalized and efficient response generation~~ last twelve questions and answers but not the search results behind earlier answers, which already contain what mattered; the current question's searches are included in full.

5.3.15 Costs and Abuse in a Public Demonstration

Problem. The demonstration is open to anyone, and every question and every uploaded page costs model usage.

Solution. Visitors enter as guest accounts that are deleted with their chats after 24 hours. Each visitor has a daily allowance of questions, enforced inside a database transaction under an advisory lock, so parallel requests cannot exceed it; usage is refunded only when no model call has returned. Uploads are turned off for visitors, and the API key has a spending limit at the provider, which caps costs even if the application's own limits were bypassed.

Document Process Algorithm

Table 6.1. *Facts Recovered and Speed of Six Text-Extraction Setups*

Setup	All facts (67)	Scanned (36)	Seconds per page	Observation
MarkItDown (text layer)	27 (40%)	0%	0.01	Nothing from scanned pages; repeats the repositories' OCR errors
Docling (standard OCR)	45 (67%)	56%	3.1	Missed most of the watermarked BOR resolutions and the typewritten page
Docling with Qwen3-VL	64 (96%)	100%	6.4	Shortened the surname "Vilela-Malabanan"
Qwen3-VL on page images	63 (94%)	97%	4.8	Shortened the same surname; omitted three dates of issue
Marker 2 (Surya OCR 2)	58 (87%)	83%	13.8	Exact names and dates; missed signatory names crossed by signatures and watermark-covered typewriting
Marker 2 with Qwen3-VL	61 (91%)	86%	22.2	Exact names and dates, plus three signatory names; eight handwriting requests failed and kept Marker's text

Note. Local models ran on a laptop GPU (Apple M4 Pro); Qwen3-VL-30B-A3B-Instruct ran through OpenRouter. Marker's time excludes the first document, which includes loading its models.

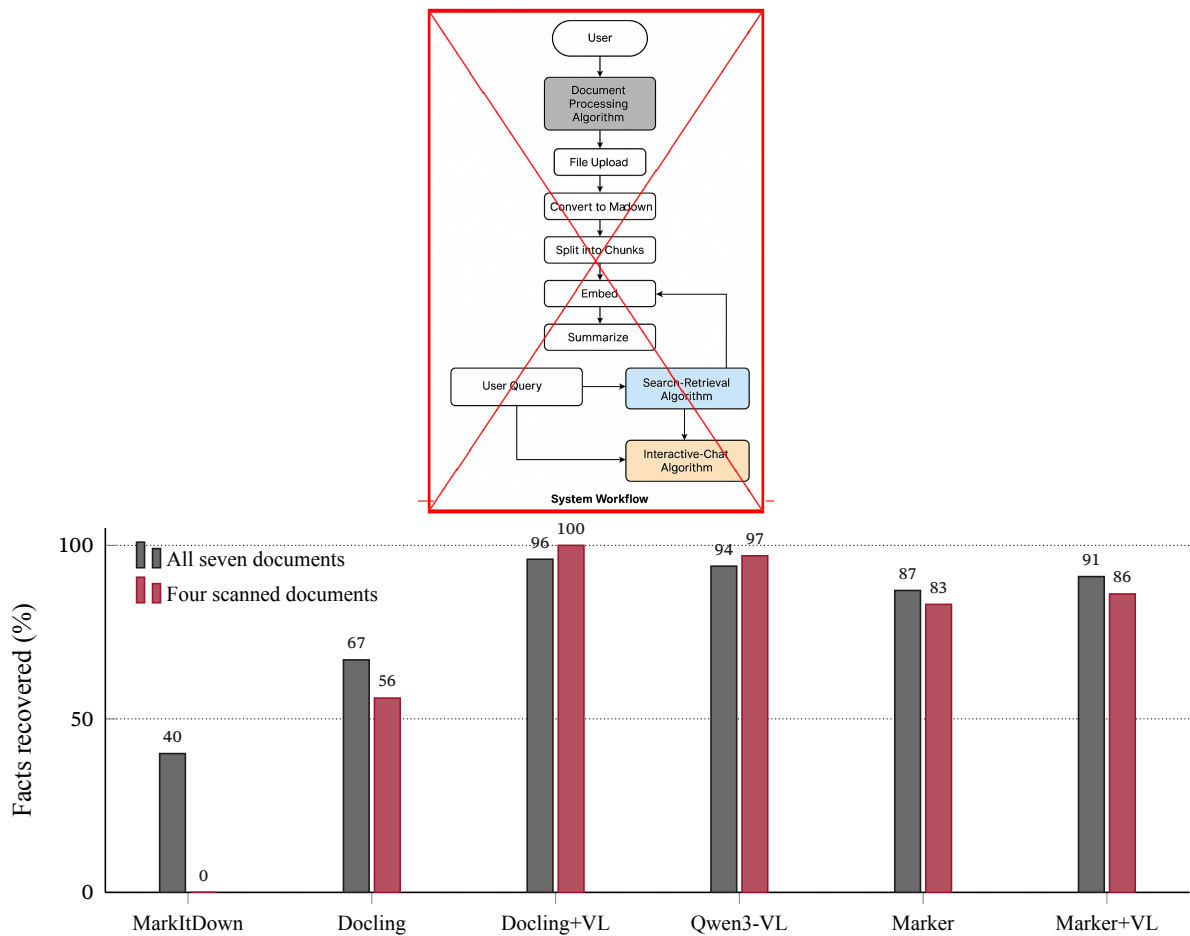


Figure 6.1. *Share of Facts Recovered by Each Text-Extraction Setup*

Surya OCR 2, a model trained only to transcribe, copied names, numbers, and dates exactly; its misses were text physically obscured by signatures and by the watermark on the 1970 resolution. Marker’s LLM mode, in which Qwen3-VL corrects only the hard regions, recovered three more facts than Marker alone—signatory names partly covered by signatures—while keeping its exact names and dates. Eight of its requests on handwriting regions failed because the model repeated blank lines until its reply was cut off; Marker then kept Surya’s own text for those regions, so a failed request costs only the improvement, never the text. CatSight AI therefore uses Marker with its LLM mode: it gives up a few points of recall compared with the whole-page vision setups in exchange for transcriptions that do not silently alter names, numbers, or dates, which are what users search for.

Marker was also the slowest setup: about 14 seconds per page on a laptop GPU, 22 seconds with its LLM mode, and about two minutes per page on the laptop’s CPU alone. This is acceptable for building a library, which happens once per document, but it is the main reason to run the OCR model on a GPU (Section 5.3.2, Table 4.4).

6.2 Processing of the Library

The deployed system processed all 48 documents with Marker 2—Surya OCR 2 with Qwen3-VL in LLM mode—and catalogued and indexed them with the open-weight models of Table 4.3. The library holds 158 pages split into 556 passages, issued between 1970 and 2024. Eleven documents failed during a brief restart of the OCR service and were resumed from the extracting stage without further intervention, as designed (Section 5.3.4); none failed otherwise. Extraction dominated the processing time: with six pages read at once, the server extracted the last 132 pages in 94 minutes, about 43 seconds per page including Marker’s LLM mode. Cataloguing took a median of 8.5 seconds per document and indexing 2.2 seconds. Each document received one to three of the 13 tags; Figure 6.2 shows how the documents are distributed across decades and tags.

System Workflow Diagram

6.3 Retrieval

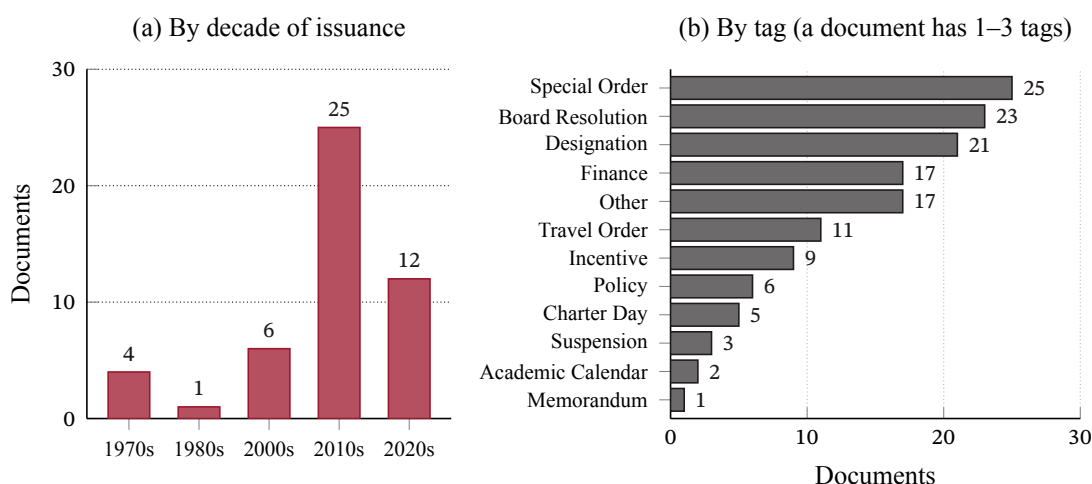


Figure 6.2. *System Workflow* The Library by Decade of Issuance and by Tag

System Functions

Figures 6.3 and 6.4 show the two kinds of matching at work. A search for “SO 1592” (Figure 6.3) finds Special Order No. 01592-IIT although the query omits the leading zero and the word “Special Order”: the keyword search matches the normalized number, and the passage is labelled an exact match. A search for “scholarship stipend for engineering technology students” (Figure 6.4) finds the 2003 resolution on grants-in-aid, whose text speaks of “Grants-in-Aid” and “School of Engineering Technology” rather than “scholarship stipend”: here the semantic search contributes. When the query is a question, the search page also writes a short cited answer above the results (Figure 6.5).

Environment and Resources

Design and Implementation Issue

Table 6.2 compares the retrieval accuracy of CatSight AI with the keyword search of IIT Docs.

This chapter discusses the design and implementation of the major data structures and algorithms used in

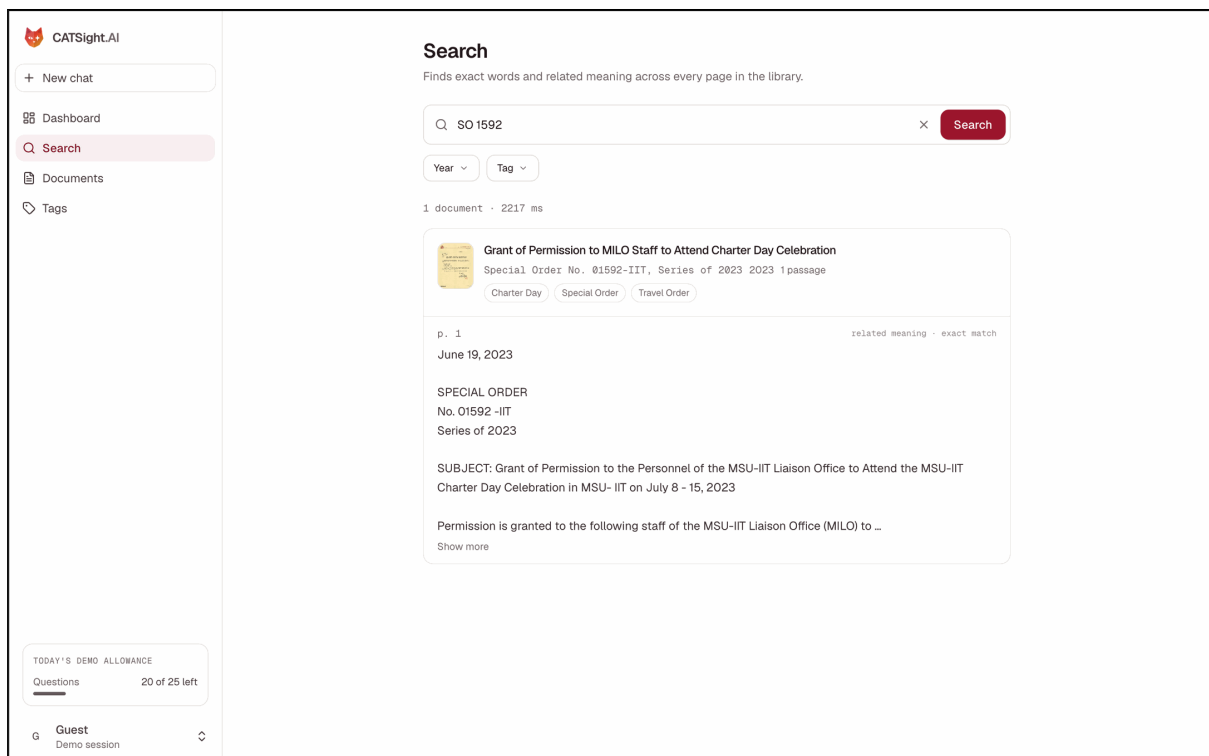


Figure 6.3. *Search by Reference Number: “SO 1592” Finds Special Order No. 01592-IIT*

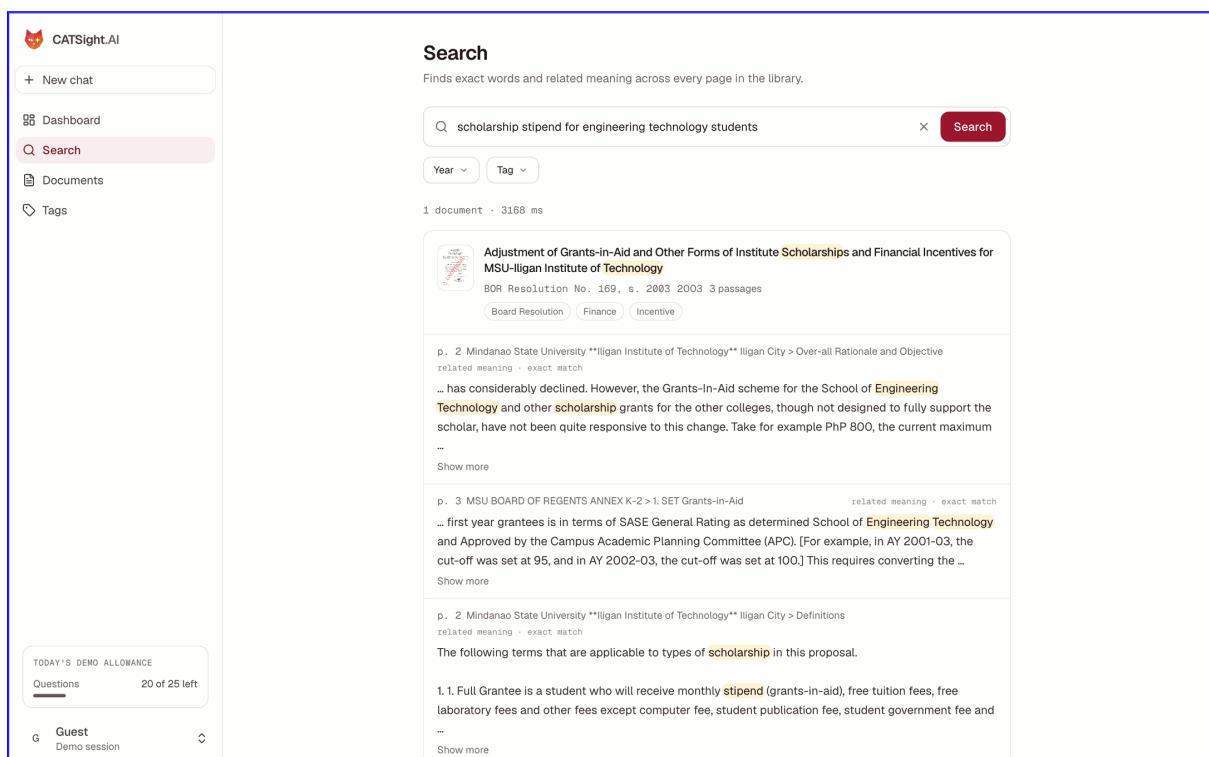


Figure 6.4. *Search by Meaning: a Paraphrased Query Finds the Grants-in-Aid Resolution*

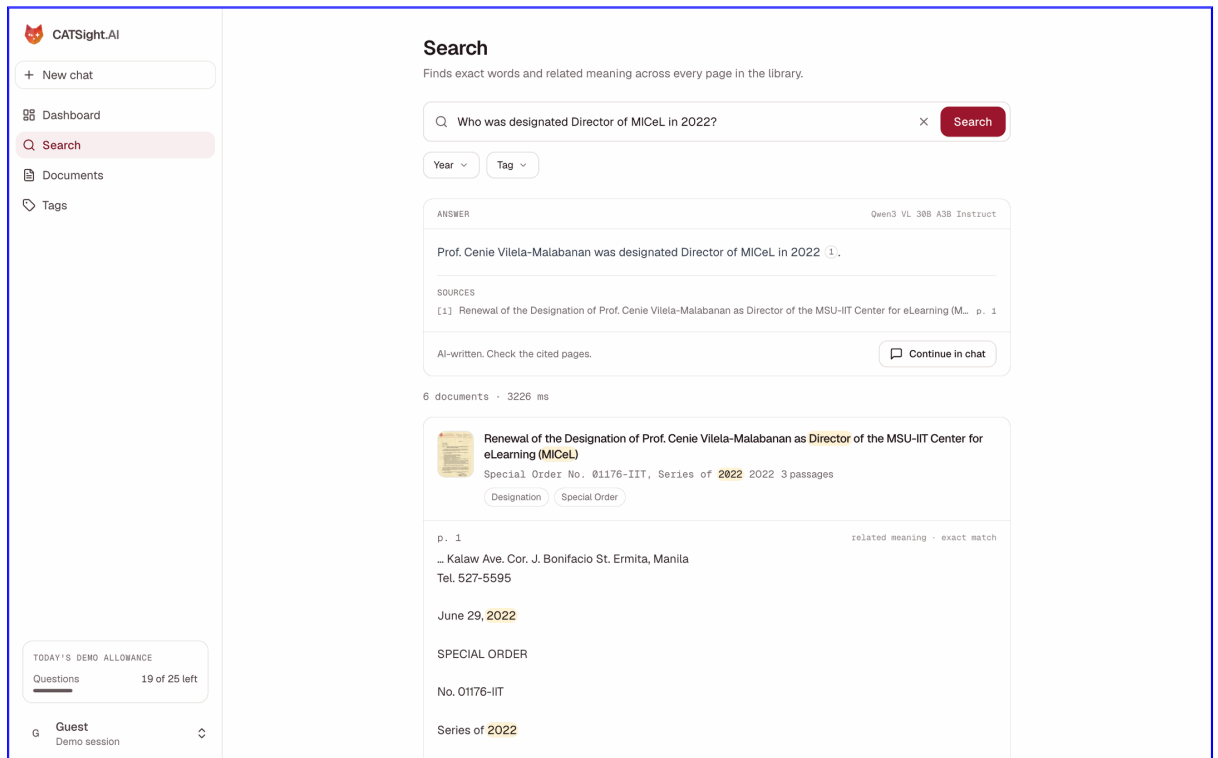


Figure 6.5. *A Question in the Search Page, with a Short Cited Answer*

Table 6.2. *Retrieval Accuracy*

System	P@5	R@5	MRR
IIT Docs (keyword search)	[TODO:]	[TODO:]	[TODO:]
CatSight AI (hybrid search)	[TODO:]	[TODO:]	[TODO:]

6.4 Chat

Asked who was designated Director of the MSU-IIT Center for eLearning in 2022 (Figure 4.21 in Chapter 4), the chat searched, named the faculty member and the effectivity date with a citation to Special Order No. 01176-IIT, and noted that the order does not state a fixed term. The follow-up in Figure 6.6 refers only to “that position”; the assistant searched for the teaching-load reduction of the MICeL director and answered with the 12-unit reduction granted under BOR Resolution No. 114, s. 2007, citing the order. In a chat limited to the 2003 grants-in-aid resolution (Figure 6.7), it read the whole resolution and answered that the new maximum monthly stipend of a full grantee is PhP 1,200.00, broken down into meals and lodging. A question in Filipino was answered in Filipino from an English resolution, with a citation (Figure 6.8).

These answers came after three fixes described in Chapter 5. In the first run on the rebuilt system, the same follow-up was answered without a search—the documents “do not specify” the reduction—the scoped question was answered with PhP 800, the old maximum quoted in the resolution’s rationale, and most answers carried no citation numbers (Sections 5.3.11–5.3.13). Neither wrong answer invented a fact, but both would have misled a reader who did not open the source. After the fixes, all eight answers in a check of eight questions carried citations, and both questions were answered correctly.

6.5 Summary Quality

Table 6.3 reports the agreement of the generated summaries with the human-written references (Section 3.3.2).

Table 6.3. *Summary Quality against Human-Written References*

	ROUGE-1	ROUGE-2	ROUGE-L	BLEU	BERTScore F ₁
CatSight AI	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

6.6 Efficiency

Table 6.4 reports response times measured on the deployed system as a guest, over the public internet, with five searches and five chat questions. A search takes about 3.3 seconds, which includes embedding the query, running both searches, and reranking, all with models reached

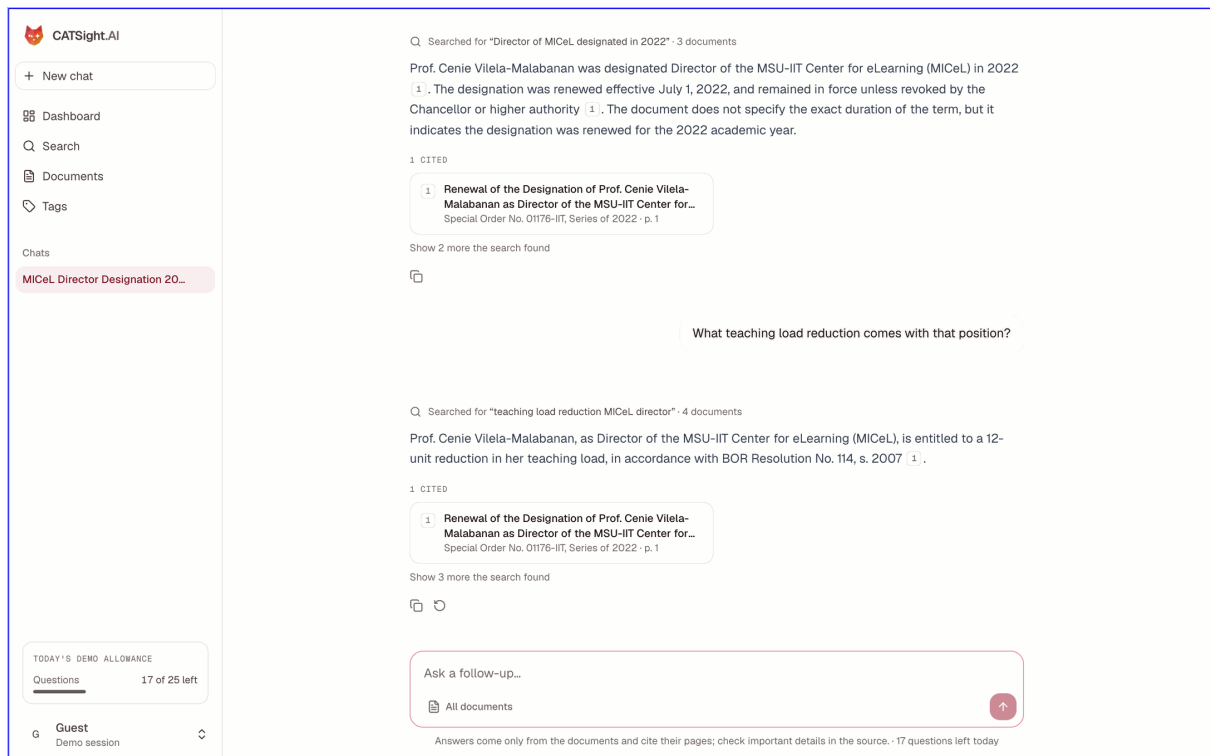


Figure 6.6. *A Follow-Up Question Answered from the Conversation*

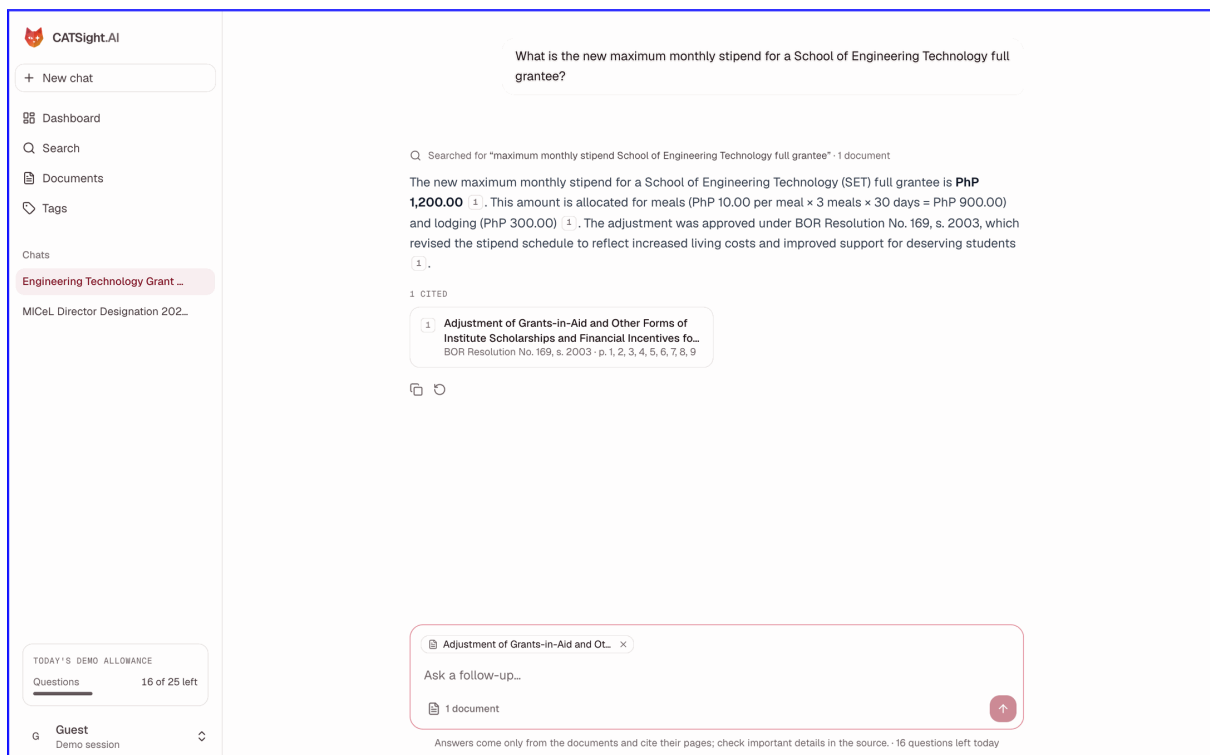


Figure 6.7. *A Chat Limited to One Document*

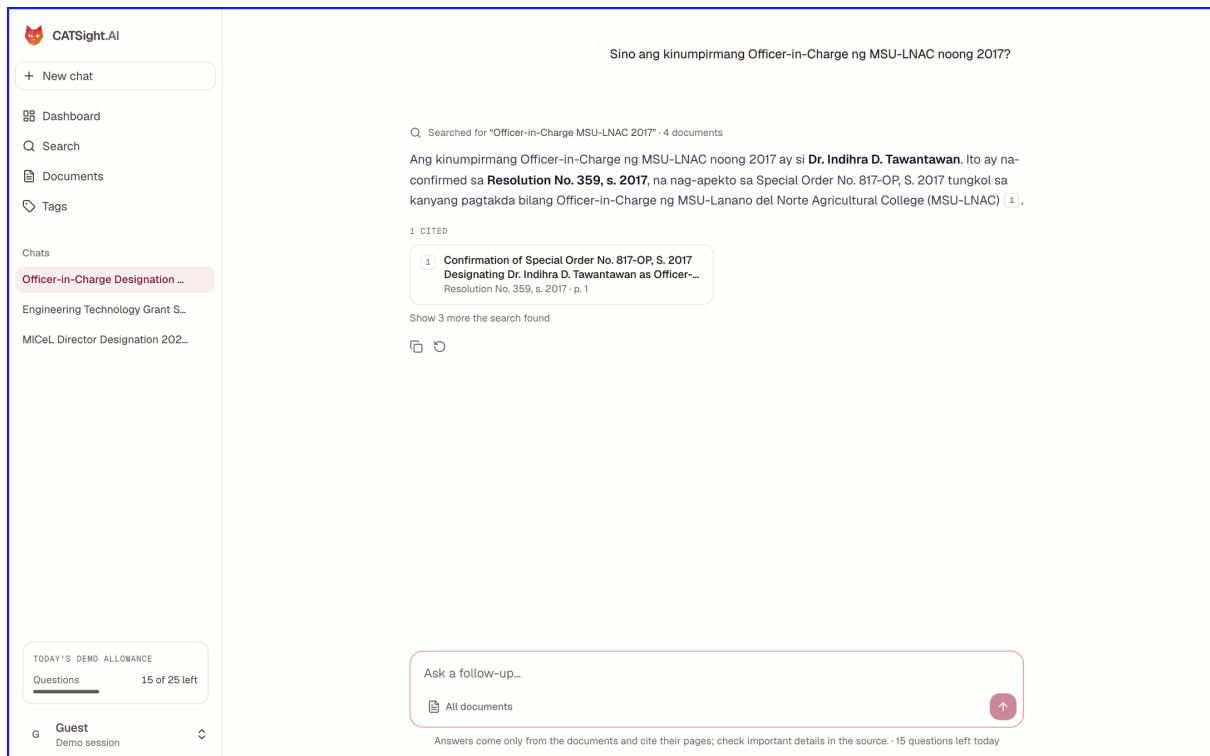


Figure 6.8. *A Question in Filipino Answered in Filipino*

over the internet. A chat answer streams: the search finishes after about 1.3 seconds, the first words of the answer appear after a median of 9.1 seconds, and the answer is complete after about 14 seconds. The hosted models' response times vary with load: earlier runs on the same day had medians as low as 5.1 seconds to the first word and 8.5 seconds to a complete answer. Building the library is slower, at about 43 seconds per page (Section 6.2), but it happens once per document.

~~This chapter discusses the design and implementation of the~~

Results and Observations

6.7 Usability

In the usability evaluation of the defended prototype, participants rated CatSight AI at an average of 77.17 on the System Usability Scale, against 50.33 for the existing keyword-based

Table 6.4. *Response Times of the Deployed System*

Measure	Median	Range
Search (5 queries)	3.3 s	2.3–5.3 s
Chat: search completed (5 questions)	1.3 s	1.1–2.4 s
Chat: first word of the answer	9.1 s	6.4–10.0 s
Chat: complete answer	13.6 s	11.0–15.0 s
Extraction of a page (132 pages)	43 s on average	

system. On the adjective scale of Bangor et al. (2009), a score in the high 70s corresponds to “good” usability and a score near 50 to “OK” or marginal usability, below the commonly cited average of 68. The difference of almost 27 points reflects what the participants could do with CatSight AI that the existing system did not allow: ask in their own words, search scanned documents, and see the source of an answer. **[TODO: Number of participants and their roles (students, faculty, staff) from the defended evaluation.]** The interface was redesigned in the rebuilt system, so the study should be repeated with it (Chapter 7).

Chapter 7 Conclusion and Recommendations

7.1 Conclusion

This study set out to develop and evaluate a content-based document retrieval system for MSU-IIT that integrates OCR, vector embeddings, and LLM-powered conversational interaction. Each specific objective (Section 1.3.2) was addressed as follows.

1. **Collection.** A corpus of 50 issuances was assembled from IIT Docs and the BOR Resolutions archive across six categories, including scanned, typewritten, watermarked, and table-heavy files (Table 4.1). Two exact duplicates were detected automatically, leaving 48 documents.
2. **Design.** The architecture (Figure 4.3) combines a resumable document pipeline, hybrid retrieval that fuses semantic and keyword search, and a single-agent chat that answers from retrieved passages with page-level citations. Every model is open-weight, so the institution can host the whole system on its own hardware (Table 4.4).
3. **Implementation.** The pipeline, search, and chat were implemented and deployed as a public demonstration. Marker 2 with Surya OCR 2 reads scanned pages, with Qwen3-VL correcting tables, forms, and handwriting; documents are catalogued zero-shot, with map-reduce for long texts; and the agent's answers stream with citations that open the source page.
4. **Evaluation.** Among the converters compared, Marker with its LLM mode and the vision-model approaches recovered by far the most facts from scanned pages, while MarkItDown recovered none (Section 6.1). In the evaluation of the defended prototype, users rated CatSight AI at 77.17 on the System Usability Scale against 50.33 for the existing system (Section 6.7). **[TODO: Add the retrieval-accuracy and summary-quality results of Sections 6.3 and 6.5 once measured.]**
5. **Analysis.** The results and the issues met during development (Chapter 5) lead to the recommendations below.

Overall, CatSight AI shows that the limitations of keyword search in MSU-IIT's repositories—semantic blindness, the exclusion of scanned files, and rigid interaction—can be addressed with open models: users ask in their own words, scanned issuances become searchable, and every answer can be checked against the page it came from.

7.2 Recommendations

1. **Complete the evaluation of the rebuilt system.** Repeat the usability study, and run the retrieval and summary evaluations of Section 4.7 with a larger set of questions and human-written reference summaries, including questions in Filipino and Cebuano.
2. **Host the models on campus.** Serve the open models on an institutional GPU server (Table 4.4) so that no document text leaves the campus network, as the Data Privacy Act may require for confidential records; the local configuration of Section 4.4.6 already supports this.
3. **Scale the library.** Ingest the complete IIT Docs and BOR archives. Beyond a few hundred thousand passages, add an approximate vector index (Section 5.3.7) and measure its effect on filtered searches.
4. **Read images and seals.** Describe photographs, diagrams, and seals with the vision model, and recognize signatures, which the current system records only as “[signature]”.
5. **Index tables on their own.** Reading a short document whole fixes questions within one document (Section 5.3.12); for questions across the whole library, index each table as a passage of its own together with the sentence that introduces it, and keep stamps that OCR turns into headings out of the section names.
6. **Learn from use.** Let users mark answers as helpful or not, and use the feedback to tune the similarity threshold, the reranker gate, and the instructions.
7. **Integrate with the repositories.** Index new issuances automatically as they are posted to IIT Docs, instead of uploading them by hand.
8. **Serve other institutions.** Apart from the cataloguing instructions and the tag list, nothing in the pipeline is specific to MSU-IIT. Let several organizations share one installation, each with its own library, members, tags, and model provider and key,

so that each bears its own usage costs and can choose local models (Section 4.4.6); every search and chat must then be restricted to the organization's own documents (Section 5.1). Other offices and universities with scanned records could then use CatSight AI without a deployment of their own.

Appendix B Personal Vitae

Arellano, Carlo P.

[TODO: Vitae from the defended manuscript (Appendix B): photo, contact, education, skills.]

Carnaje, Michael James M.

[TODO: Vitae from the defended manuscript (Appendix B): photo, contact, education, skills.]

Lavesores, Fulgent Kvasir E.

[TODO: Vitae from the defended manuscript (Appendix B): photo, contact, education, skills.]